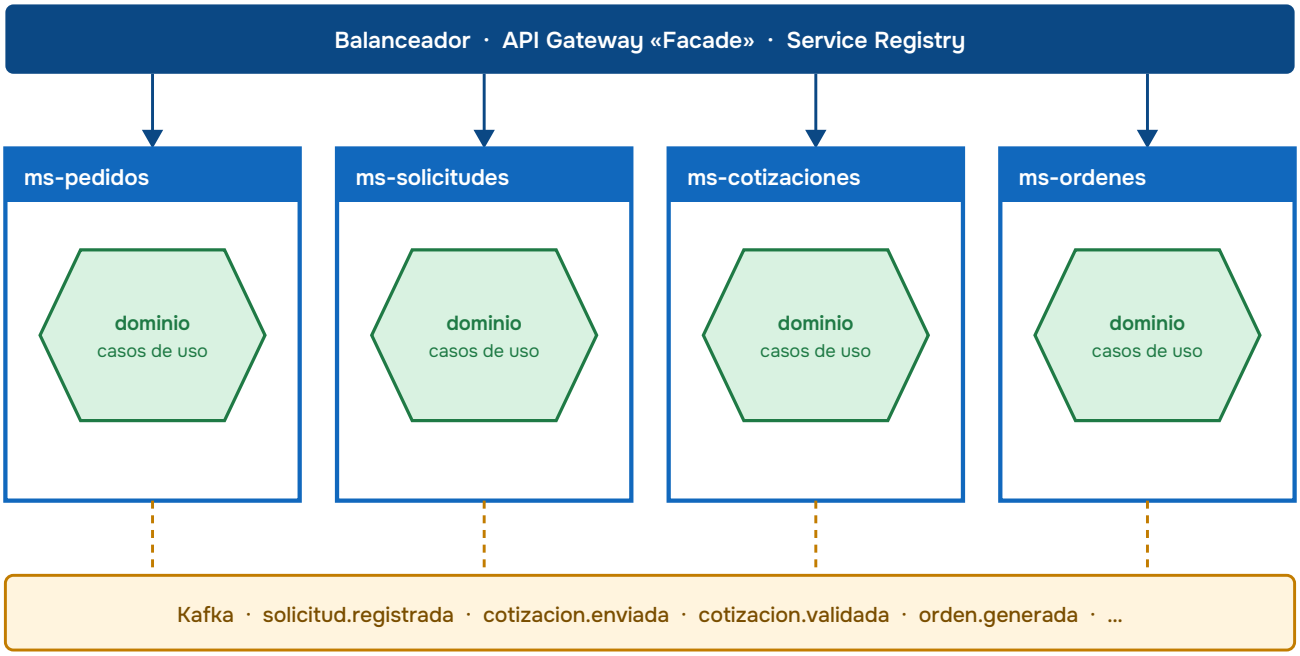
Sistema de Abastecimiento Konrad Gourmet

Por qué combinar microservicios, arquitectura orientada a eventos y arquitectura hexagonal, y cómo encaja cada una con las 23 ventanas del prototipo y con el código.



- Microservicios: cómo se parte y se despliega
- Orientada a eventos: cómo se comunican los roles
- Hexagonal: cómo se construye cada servicio

Fundación Universitaria Konrad Lorenz · Programa de Ingeniería de Sistemas

Arquitectura de Software · Profesor Carlos Andrés López · Octubre de 2026

Acompaña a Arquitectura_Konrad_Gourmet.drawio (8 páginas) y al esqueleto Java del paquete co.edu.konradlorenz.

Contenido

1 Resumen de la decisión . . . 3

2 Método de análisis . . . 4

3 Requerimientos no funcionales y decisiones . . . 5

4 Estilos evaluados . . . 6

5 Microservicios . . . 7

6 Arquitectura orientada a eventos . . . 9

7 Arquitectura hexagonal . . . 10

8 Estilos de apoyo . . . 12

9 El mapa integrado y su notación . . . 13

10 Arquitectura por flujo de ventanas . . . 20

11 Eventos del sistema . . . 21

12 Patrones de diseño en el nivel de código . . . 22

13 Del diagrama al código . . . 26

14 Contexto y despliegue . . . 27

15 Trazabilidad de las 23 ventanas . . . 30

16 Riesgos y mitigaciones . . . 31

17 Supuestos . . . 31

18 Declaración de uso de inteligencia artificial . . . 32

19 Referencias . . . 33

Anexo A · Nivel de código completo . . . 34

Este documento acompaña dos entregables: el archivo **Arquitectura_Konrad_Gourmet.drawio**, con ocho páginas (mapa integrado, contexto C4, cinco páginas de nivel de código y despliegue), y el **esqueleto Java** que implementa las mismas clases. Las figuras de este PDF son capturas de ese archivo; para editar o ampliar, ábralo en draw.io.

1 Resumen de la decisión

Konrad Gourmet necesita que cuatro roles operativos —mesero, jefe de cocina, auxiliar de compras y director de compras— trabajen sobre los mismos datos desde todas las sucursales, con 200.000 usuarios concurrentes, 1.000 transacciones por segundo y una disponibilidad del 99,7 %. Ningún estilo arquitectónico cumple solo esas restricciones, así que la propuesta combina tres, cada uno en el nivel donde resuelve un problema distinto.

Arquitectura	Dónde vive	Qué decide	Requerimientos que la exigen	Dónde se ve
Microservicios	Nivel 2 del C4 (contenedores)	Once servicios desplegables, cada uno con su base de datos, detrás de un API Gateway	200.000 usuarios, 1.000 TPS, 99,7 %, computación distribuida, crecimiento del 200 %	Mapa (cajas azules) y página de despliegue
Orientada a eventos	Entre contenedores	Comunicación asíncrona entre roles y procesos automáticos por medio de diez tópicos	Correo certificado, alarma del 25 %, auditoría de cada CRUD, tendencias, desacople ante caídas	Mapa (banda ámbar y flechas punteadas)
Hexagonal	Niveles 3 y 4 del C4, dentro de cada servicio	Dominio aislado de la base de datos, del SOAP, del SMTP y del PDF	Cambio fácil de motor de BD, integración SOAP, tecnología libre, mantenimiento	Mapa (hexágonos) y páginas 4a–4e

Las tres se leen juntas en la página 1 del archivo .drawio: las cajas azules son los microservicios, la banda ámbar es el broker de eventos y los hexágonos verdes son el interior de cada servicio. A su alrededor aparecen cinco estilos y mecanismos de apoyo que no compiten con los tres principales sino que los completan: cliente-servidor con REST entre la SPA y el API Gateway, SOA con SOAP en la frontera con los sistemas externos, OAuth 2.0 con JWT para la seguridad, un Service Registry para descubrir instancias y un balanceador de carga activo-activo.

El diagrama no es un dibujo aparte del código. Las clases del nivel 4 y el esqueleto Java salen del mismo modelo, cada uno de los 23 mockups está asignado a un controlador y un método concretos, y una demostración ejecutable reproduce los valores de la ventana 19 (cotización COT-018, diferencia de +4,81 % frente al precio del Ministerio, estado OPCIONADA).

> **«Un diseño no es bueno o malo en sí mismo, sino con respecto a las restricciones que se le imponen.»**
>
> Profesor Carlos Andrés López, sesión 10. Por eso cada decisión de este documento se justifica contra un requerimiento no funcional del enunciado.

La sección 3 recorre los requerimientos no funcionales uno por uno; la 4 descarta los estilos vistos en clase que no encajan; las secciones 5 a 7 desarrollan cada arquitectura con la plantilla de clase (esquema, lineamientos, aplicabilidad, ventajas y desventajas, ejemplo); la 9 explica el mapa y su notación; la 10 asigna una arquitectura dominante a cada flujo de ventanas; y las secciones 12 y 13 muestran cómo el diagrama de clases y el código implementan los patrones de diseño.

2 Método de análisis

Fuentes

Se analizaron tres fuentes. La primera es el enunciado del proyecto, con sus requerimientos funcionales y diecinueve requerimientos no funcionales agrupados en seguridad, desempeño, interfaz gráfica, almacenamiento, mantenimiento, tecnología e integración. La segunda son las 23 ventanas del prototipo —quince de inicio de sesión, mesero, jefe de cocina y auxiliar de compras, y ocho del director de compras— junto con el flujo web que las conecta botón por botón. La tercera son las nueve sesiones de clase, en las que los grupos expusieron los estilos de arquitectura y el profesor presentó el modelo C4 y los patrones de diseño.

Criterios tomados de las clases

El profesor dejó criterios explícitos que guiaron cada decisión. Un diseño se juzga frente a sus restricciones, así que cada estilo elegido debe responder a un requerimiento no funcional concreto. Los estilos se combinan: al hacer zoom sobre una caja se puede usar otro estilo por dentro. La tecnología se decide lo más tarde posible para mantener flexibilidad, y las flechas deben llevar su protocolo. El modelo C4 es obligatorio en sus cuatro niveles —contexto, contenedores, componentes y código— y en el nivel de código, con diagrama de clases UML, debe verse dónde va cada patrón; los diagramas dinámico y de despliegue son complementarios. Cada estilo se documenta con la plantilla que usaron los grupos: esquema, lineamientos, políticas y reglas, aplicabilidad, ventajas y desventajas, y un ejemplo.

Cómo se leyeron las ventanas

De cada ventana se registró el rol, la acción principal (el botón que cambia datos), la información que crea o consulta y si esa acción cambia lo que ve otro rol. Esta última pregunta fue decisiva. Cuando un clic se resuelve dentro del mismo rol y el usuario espera la respuesta —registrar un pedido en la ventana 03 y ver PED-0251 en la 04—, la comunicación es síncrona. Cuando el clic de un rol modifica la pantalla de otro —el auxiliar envía una cotización en la 13 y la solicitud del jefe pasa a EN TRÁMITE en la 10—, esa costura se modeló como evento. Así aparecieron las fronteras entre servicios y los diez tópicos del broker.

3 Requerimientos no funcionales y decisiones

La tabla recorre los requerimientos no funcionales del enunciado (algunos agrupados) y muestra qué decisión produce cada uno, qué arquitectura la asume y dónde se ve en el diagrama. Los colores son los mismos del mapa: azul para microservicios, ámbar para eventos, verde para hexagonal y gris para los estilos de apoyo.

Requerimiento	Decisión	Arquitectura	Dónde se ve
200.000 usuarios concurrentes y 1.000 TPS de compras	Servicios sin estado detrás de un balanceador; solo los que reciben más carga (pedidos, cotizaciones, identidad) escalan en horizontal	**Microservicios**	Despliegue: ms-pedidos de 4 a 20 réplicas
Computación distribuida	Once servicios independientes que se registran en un Service Registry y se descubren en tiempo de ejecución	**Microservicios**	Mapa: Service Registry [Eureka]
Alta disponibilidad del 99,7 % (unas 26 horas de caída al año)	Sin punto único de falla: balanceador activo-activo, tres réplicas del gateway, Kafka con factor de réplica 3, clúster en tres zonas. Si un servicio cae, los demás siguen y sus eventos esperan en el broker	**Microservicios** + **eventos**	Despliegue
Centro de datos alterno	Región B en espera tibia con réplica asíncrona de bases y tópicos; conmutación por DNS	**Apoyo · despliegue**	Despliegue, borde magenta
Autenticación, autorización, roles y permisos	ms-identidad emite un JWT (OAuth 2.0); el gateway lo valida en cada petición; el menú lateral se arma por rol	**Microservicios** + **OAuth 2.0**	V01; página 4a (Composite)
HTTPS en toda comunicación	TLS 1.3 hacia el usuario, mTLS entre servicios y JDBC con TLS	**Apoyo**	Protocolo en cada flecha
Contraseña de 8 o más caracteres con mayúscula, minúscula y número, almacenada cifrada	Regla en el dominio de identidad; hash BCrypt detrás del puerto CifradorPort	**Hexagonal**	Adaptador BCrypt de ms-identidad
Correo certificado con estampado cronológico	Decoradores sobre el puerto Correo: el certificado y el sello de tiempo se apilan sobre el adaptador SMTP; los correos se disparan por eventos	**Hexagonal** + **eventos**	Página 4d (Decorator)
Imagen corporativa parametrizable	Tema y logos en ParametrosSistema (ms-administracion); la SPA los carga al iniciar, sin redesplegar	**Microservicios**	ms-administracion (Singleton)
Diseño responsive para móviles	Una sola SPA React + TypeScript responsive para web y móvil	**Cliente-servidor**	Contenedor SPA del mapa
Crecimiento del 200 % en documentos, archivos y datos	PDF y archivos de carga en un almacén de objetos; cada base crece y se dimensiona por separado	**Microservicios**	Despliegue, capa de datos
Backup diario	Copia completa nocturna más registros WAL continuos, con retención de 30 días	**Apoyo · despliegue**	Despliegue, respaldo diario
Cambio fácil de motor de base de datos	Cada repositorio es un puerto; el motor vive en un adaptador JPA que se reemplaza sin tocar el dominio	**Hexagonal**	Adaptadores *JpaAdapter
Auditoría de cada CRUD con acción, usuario, fecha y hora	Cada servicio publica auditoria.evento; ms-administracion lo consume y lo guarda en su bitácora	**Eventos**	Tópico auditoria.evento
Registro de cada error en un log	Logs centralizados (ELK) con trazas OpenTelemetry; consulta desde ms-administracion	**Apoyo**	Despliegue, observabilidad
Tecnología libre de licenciamiento y versiones estables	Java 21, Spring Boot 3, PostgreSQL, Apache Kafka, Redis, React, Kubernetes: todo de código abierto	**Todas**	Etiquetas [tecnología] del mapa
Exponer servicios a sistemas externos por SOAP	ms-integracion concentra la frontera SOA: publica un endpoint SOAP y consume el servicio web del Ministerio de Comercio	**SOA (apoyo)** + **Hexagonal**	Endpoint SOAP de ms-integracion

4 Estilos evaluados

En las exposiciones de clase se presentaron quince estilos, mecanismos y componentes. Todos se evaluaron contra las restricciones del enunciado: tres se eligieron como arquitectura principal, seis quedaron como apoyo y seis se descartaron o se absorbieron en otro.

Estilo o mecanismo	Veredicto	Razón para Konrad Gourmet
Microservicios	**Elegida · estructura**	Permite escalar solo lo que recibe carga, aislar fallas entre roles y desplegar cada parte por separado. Es la única opción vista que responde a 1.000 TPS y al 99,7 % sin escalar todo el sistema a la vez.
Orientada a eventos	**Elegida · comunicación**	El enunciado está lleno de efectos secundarios (correos, alarma del 25 %, auditoría, paso a EN TRÁMITE, KPI, tendencias) que no deben bloquear al usuario ni acoplar servicios.
Hexagonal (puertos y adaptadores)	**Elegida · interior**	Aísla el dominio de la base de datos, del SOAP del Ministerio y del SMTP, que son justamente los puntos que el enunciado pide poder cambiar o integrar.
Limpia (Clean Architecture)	Absorbida en la hexagonal	Persigue lo mismo: dependencias hacia el dominio. Aplicar ambas duplicaría capas; se conserva su regla de dependencia y la separación entre casos de uso y entidades.
Capas (monolito)	Descartada como estilo global	Un solo despliegue obliga a escalar todo para atender el pico de pedidos, y una falla detiene a los cuatro roles. La idea de capas sigue viva dentro de cada hexágono.
Cliente-servidor	**Apoyo**	La SPA es el cliente y el API Gateway el servidor; es la relación natural de una aplicación web responsive.
REST	**Apoyo**	Contrato síncrono para las 23 ventanas y para las dos consultas entre servicios que necesitan respuesta inmediata.
SOA con SOAP	**Apoyo en la frontera**	El requerimiento de integración obliga a exponer servicios por SOAP y el Ministerio publica un servicio web. Se limita a ms-integracion para que el resto del sistema no dependa de SOAP.
ESB central	Descartado	Un bus de integración central orquestaría a todos los servicios y se volvería cuello de botella y punto único de falla, riesgo que señaló el propio grupo expositor. Kafka da el desacople sin lógica central.
Peer-to-peer (P2P)	Descartado	No hay pares equivalentes: pedidos, inventario y compras deben estar centralizados, auditados y protegidos por roles.
Polling	Descartado entre servicios	Preguntar cada cierto tiempo si algo cambió consume transacciones sin aportar. Solo se conserva la tarea programada diaria de inventario, que el enunciado pide así.
Webhooks	Descartado internamente	Sirven para avisar a terceros por HTTP, pero dentro del sistema Kafka ofrece persistencia, reintentos y varios consumidores. Queda como opción futura para avisar a proveedores.
Balanceador de carga	**Apoyo**	Par activo-activo de capa 7 delante del gateway, con chequeos de salud cada 5 segundos.
Service Registry	**Apoyo**	Eureka mantiene la lista de instancias vivas para que el gateway enrute aunque el autoescalado cambie las réplicas.
API Gateway	**Apoyo · Facade del sistema**	Punto único de entrada: valida el JWT y enruta por prefijo de URL hacia cada servicio, ocultando a la SPA cuántos servicios hay detrás.

ARQUITECTURA 1 · ESTRUCTURA DEL SISTEMA

5 Microservicios

Define cómo se parte y se despliega el sistema. Vive en el nivel 2 del modelo C4 (contenedores) y responde a los requerimientos de desempeño, disponibilidad y almacenamiento.

Esquema

La SPA envía todas las peticiones por HTTPS al balanceador de carga, que las reparte entre tres réplicas del API Gateway. El gateway valida el JWT emitido por ms-identidad, consulta el Service Registry y enruta por prefijo (/api/pedidos/**, /api/cotizaciones/**…) a una instancia sana del servicio. Cada uno de los once microservicios es un contenedor Spring Boot con su propia base PostgreSQL, que ningún otro servicio lee directamente. En el mapa integrado es la fila de once cajas azules debajo del gateway.

Lineamientos, políticas y reglas

El lineamiento es que cada servicio sea dueño de un subdominio del negocio y de sus datos. Las políticas son una base de datos por servicio, servicios sin estado (la sesión viaja en el JWT) para poder replicarlos, comunicación síncrona solo por REST a través del gateway o de un cliente declarado, y despliegue independiente con versión de API. Las reglas concretas: ningún servicio consulta las tablas de otro; ninguna petición del usuario encadena más de dos llamadas síncronas entre servicios; cada servicio expone un chequeo de salud para el balanceador y se registra en Eureka al arrancar; y cada servicio publica un evento de auditoría en cada CRUD.

Aplicabilidad en Konrad Gourmet

Los cortes siguen el ciclo de vida de los datos y los roles que los tocan: el pedido lo crea el mesero, el plato y el inventario los maneja el jefe de cocina, la solicitud pasa del jefe al auxiliar, la cotización del auxiliar al director y la orden la emite el director. A eso se suman cinco servicios transversales (identidad, notificaciones, administración, analítica e integración). La carga no es pareja: los meseros de todas las sucursales registran pedidos todo el día y las 1.000 TPS de compras se concentran en pedidos y cotizaciones, mientras que el registro de platos es esporádico. Con microservicios, ms-pedidos escala de 4 a 20 réplicas y ms-menu se queda en 2.

Servicio	Responsabilidad	Ventanas	Ruta en el gateway
ms-identidad	OAuth 2.0 / OIDC, usuarios, roles, permisos y menú por rol	V01	`/api/auth/**`
ms-pedidos	Pedidos por mesa; escala horizontal en horas pico	V02–V04	`/api/pedidos/**`
ms-menu	Platos, recetas, categorías y productos	V06	`/api/platos/**`
ms-inventario	Existencias por sucursal, descuento diario y umbral del 25 %	V05 · V07	`/api/inventario/**`
ms-solicitudes	Solicitudes de alimentos y sus estados	V08–V12	`/api/solicitudes/**`
ms-cotizaciones	Solicitud de cotización, PDF, registro de valores y validación	V13–V20	`/api/cotizaciones/**`
ms-ordenes	Órdenes de compra desde cotizaciones OPCIONADAS	V21–V23	`/api/ordenes-compra/**`
ms-administracion	Parametrización, imagen corporativa, auditoría y logs	parametrización y auditoría (sin mockup)	`/api/admin/**`
ms-analitica	Tablero BAM (KPI) y detección de tendencias	tablero BAM (sin mockup)	`/api/bam/**`
ms-notificaciones	Correos certificados con sello de tiempo y alarmas en la app	correos de V09, V14 y V22; alarma de V05	`/api/notificaciones/**`

Servicio	Responsabilidad	Ventanas	Ruta en el gateway
ms-integracion	Frontera SOA: consume el WS del Ministerio y expone servicios SOAP	precio estándar en V13	`/api/integracion/**`

Ventajas

Escalado selectivo de los servicios con más carga; aislamiento de fallas (si ms-analitica cae, el mesero sigue registrando pedidos); despliegue independiente, de modo que un cambio en la validación de cotizaciones no exige redesplegar pedidos; cada base de datos puede cambiar de motor o de tamaño por separado, lo que también responde al crecimiento del 200 %; y equipos que pueden trabajar en paralelo sobre servicios distintos.

Desventajas y cómo se mitigan

La operación se vuelve más compleja (once despliegues), lo que se mitiga con Kubernetes, infraestructura como código y observabilidad centralizada. Aparece la consistencia eventual entre servicios, que se maneja con estados explícitos (PENDIENTE, EN TRÁMITE, ENVIADA, OPCIONADA) y eventos idempotentes. La red añade latencia, pero ninguna ventana necesita más de una llamada síncrona entre servicios. Las consultas que cruzan datos de varios servicios, como los KPI, no leen bases ajenas: ms-analitica arma su propio almacén con los eventos.

Ejemplo en las ventanas

En la ventana 21 el director pulsa «Generar orden de compra». La SPA envía POST /api/ordenes-compra; el gateway lo enruta a ms-ordenes, que pide por REST los datos de la cotización OPCIONADA a ms-cotizaciones (CotizacionRestClient, síncrono porque el director espera la respuesta), guarda la orden en su propia base y responde 201 con OC-0012, que aparece en la ventana 22. Si ms-analitica o ms-notificaciones estuvieran caídos en ese momento, la orden se genera igual.

ARQUITECTURA 2 · COMUNICACIÓN ENTRE ROLES

6 Arquitectura orientada a eventos

Define cómo se avisan los servicios entre sí sin esperarse. Vive entre los contenedores y responde a los requerimientos de correo certificado, alarma de inventario, auditoría y disponibilidad.

Esquema

Apache Kafka con tres brokers y diez tópicos. Cada servicio publica hechos de negocio en pasado (pedido.registrado, cotizacion.validada) a través del puerto EventPublisherPort, y los servicios interesados se suscriben con listeners. Es coreografía, no orquestación: nadie dirige el proceso y cada consumidor decide qué hacer con el evento. En el mapa es la banda ámbar inferior con sus flechas punteadas.

Lineamientos, políticas y reglas

El lineamiento es el desacoplamiento temporal: el productor no sabe quién consume ni espera respuesta. Las políticas son eventos inmutables con identificador, tipo, fecha y usuario; entrega al menos una vez; consumidores idempotentes que recuerdan los identificadores ya procesados; orden garantizado por clave (número de solicitud o de cotización) dentro de una partición; y una cola de errores (DLQ) para los mensajes que fallan varias veces. Las reglas: un tópico por hecho de negocio con el nombre entidad.verbo-en-pasado; ningún evento lleva contraseñas; retención de siete días; y el esquema de cada evento se versiona.

Aplicabilidad en Konrad Gourmet

El enunciado describe muchos efectos que ocurren después de la acción del usuario. Al generar una cotización hay que enviar el PDF por correo al proveedor y revisar si la solicitud del jefe ya pasa a EN TRÁMITE. Después de cada CRUD hay que auditar. Cada pedido alimenta el descuento de inventario y los KPI. Cuando una existencia llega al 25 % hay que mostrar una alarma y enviar un correo, y las tendencias generan correos de promociones. Si todo eso fuera síncrono, la ventana 13 esperaría al servidor de correo y una caída del correo impediría cotizar.

Ventajas

El usuario recibe respuesta inmediata porque los efectos secundarios ocurren en segundo plano. Un servicio caído no arrastra a los demás: sus eventos esperan en Kafka hasta que vuelve. Agregar un consumidor nuevo, como ms-analitica, no obliga a tocar al productor. Y el propio flujo de eventos es una bitácora que facilita la auditoría pedida por el enunciado.

Desventajas y cómo se mitigan

La consistencia es eventual: el jefe puede ver PENDIENTE durante unos segundos después de que el auxiliar envió la cotización, por eso la ventana 10 muestra la fecha de la última actualización. Puede haber mensajes duplicados, que se neutralizan con idempotencia. La depuración es más difícil, así que cada evento lleva un identificador de correlación y se traza con OpenTelemetry. Kafka es una pieza más que debe estar disponible, y por eso corre con tres brokers y factor de réplica 3.

Ejemplo en las ventanas

En la ventana 13 el auxiliar pulsa «Enviar solicitud de cotización». ms-cotizaciones guarda la cotización en estado ENVIADA, genera el PDF, publica cotizacion.enviada y responde 201, con lo que aparece la ventana 14. En paralelo, ms-notificaciones consume el evento y envía el PDF al proveedor por correo certificado, y ms-solicitudes lo consume y, si todos los productos de la solicitud ya tienen cotización, la pasa a EN TRÁMITE y publica solicitud.en-tramite, que el jefe ve en la ventana 10. ms-cotizaciones no conoce a ninguno de los dos consumidores.

ARQUITECTURA 3 · INTERIOR DE CADA SERVICIO

7 Arquitectura hexagonal

Define cómo se construye cada microservicio por dentro. Vive en los niveles 3 y 4 del modelo C4 (componentes y código) y responde a los requerimientos de cambio de motor de base de datos, integración SOAP, correo certificado y mantenimiento.

Esquema

Cada servicio pone el dominio y los casos de uso en el centro (el hexágono verde del mapa), los puertos como interfaces en el borde y los adaptadores afuera. Arriba están los adaptadores de entrada, que conducen el núcleo (controlador REST, tarea programada, listener de Kafka); abajo, los de salida, que implementan puertos (JPA, publicador de Kafka, SMTP, SOAP, PDF). En el código, cada capa es un paquete:

Paquete	Contiene	Ejemplo en ms-cotizaciones
`dominio`	Entidades, objetos de valor y reglas de negocio, sin frameworks	Cotizacion, ItemCotizacion, ReglaValidacion y sus tres reglas
`puertos.entrada`	Interfaces de los casos de uso que el exterior invoca	GestionarCotizacionUseCase
`aplicacion`	Implementación de los casos de uso; coordina dominio y puertos	CotizacionService, GenerarCotizacionFacade
`puertos.salida`	Interfaces que el dominio necesita del exterior	CotizacionRepositoryPort, PrecioSugeridoPort, GeneradorPdfPort
`adaptadores.entrada`	Conducen el núcleo: REST, listeners de Kafka, tareas programadas	CotizacionRestController
`adaptadores.salida`	Implementan los puertos con una tecnología concreta	CotizacionJpaAdapter, PdfCotizacionAdapter, PrecioSugeridoRestAdapter
`eventos`	Hechos de negocio que el servicio publica	CotizacionEnviadaEvent, CotizacionValidadaEvent

Lineamientos, políticas y reglas

El lineamiento es que las dependencias apunten siempre hacia el dominio. Las políticas: el dominio no importa Spring, JPA, Kafka ni SOAP; toda salida al mundo exterior pasa por un puerto que el propio núcleo define; y los adaptadores se conectan al arrancar, por inyección de dependencias. Las reglas: un caso de uso por puerto de entrada; los controladores no contienen reglas de negocio; y el dominio se prueba con adaptadores falsos en memoria, sin base de datos ni broker.

Aplicabilidad en Konrad Gourmet

Tres requerimientos piden aislar tecnología. El cambio fácil de motor de base de datos se resuelve con un puerto de repositorio y un adaptador JPA por servicio. La integración con el Ministerio pasa por el puerto PrecioReferenciaPort, cuyo adaptador habla SOAP. El correo certificado pasa por el puerto Correo, cuyo adaptador habla SMTP. Además, el profesor pidió decidir la tecnología lo más tarde posible, y la hexagonal deja esa decisión encerrada en los adaptadores.

Ventajas

El dominio se prueba sin infraestructura: la demostración del esqueleto Java corre sin base de datos ni Kafka. Cambiar PostgreSQL por otro motor afecta un solo adaptador por servicio. Las reglas del negocio —el 25 % del inventario, el +25 % y el -50 % de las cotizaciones— quedan en un único lugar. Y el mismo caso de uso puede invocarlo un controlador REST, un listener o la tarea diaria.

Desventajas y cómo se mitigan

Hay más clases e interfaces que en un CRUD directo; para no repetir trabajo, la librería común compartido trae EventPublisherPort, EventoDominio y AuditoriaEvent. Se necesita traducir entre entidades de dominio y de persistencia, trabajo que solo existe en los adaptadores JPA. Y existe el riesgo de sobrediseñar servicios pequeños como ms-menu; se acepta a cambio de que los once servicios tengan la misma estructura y cualquier desarrollador sepa dónde buscar.

Ejemplo en las ventanas

En la ventana 13, CotizacionRestController (adaptador de entrada) invoca el caso de uso, que delega en GenerarCotizacionFacade. La fachada pide el precio estándar de cada producto por PrecioSugeridoPort; su adaptador llega por REST a ms-integracion, donde PrecioReferenciaCacheProxy responde desde la caché o delega en MinisterioComercioAdapter, que habla SOAP con el Ministerio. En la ventana 19, CotizacionService aplica la cadena de reglas del dominio a esos valores y guarda por CotizacionRepositoryPort. Para COT-018 el precio estándar es $520.000 y el del proveedor $545.000: la diferencia es +4,81 %, dentro del rango, y la cotización queda OPCIONADA, como en el mockup.

8 Estilos de apoyo

Los tres estilos principales se apoyan en cinco mecanismos que completan el diseño. Ninguno define la forma del sistema, pero sin ellos no se cumplirían varios requerimientos.

Cliente-servidor con REST

La SPA, construida con React y TypeScript y servida por Nginx detrás de una CDN, es el único cliente. Es responsive, así que la misma base de código sirve para escritorio y móvil, y consume REST con JSON sobre HTTPS. Cada ventana corresponde a un endpoint concreto (sección 15). Entre servicios solo hay dos llamadas síncronas: ms-ordenes consulta a ms-cotizaciones al generar la orden y ms-cotizaciones consulta a ms-integracion el precio estándar.

SOA con SOAP en la frontera

ms-integracion es el único servicio que habla SOAP. Hacia afuera publica AbastecimientoSoapEndpoint, con su WSDL, para los sistemas externos que el requerimiento de integración menciona. Hacia el Ministerio de Comercio usa MinisterioComercioAdapter, protegido por PrecioReferenciaCacheProxy con una caché en Redis. El resto del sistema le habla por REST, de modo que si mañana el Ministerio publica una API REST, solo cambia un adaptador.

Seguridad con OAuth 2.0 y JWT

En la ventana 01 la SPA envía POST /api/auth/token. ms-identidad busca al usuario, compara la contraseña con su hash BCrypt y emite un JWT con el rol y el menú lateral, que arma la subclase de Usuario correspondiente (Factory Method) como un árbol de opciones (Composite). El gateway valida la firma del token en cada petición, cada servicio comprueba el rol en sus endpoints y las llamadas internas usan mTLS.

Balanceador, API Gateway y Service Registry

Un par de balanceadores de capa 7 en activo-activo reparte el tráfico entre tres réplicas del gateway (Spring Cloud Gateway), que actúa como Facade del sistema: la SPA ve una sola dirección aunque detrás haya once servicios. El gateway pregunta al Service Registry (Eureka, con latido cada 30 segundos) qué instancias están vivas, lo que permite que el autoescalado agregue o quite réplicas sin reconfigurar nada.

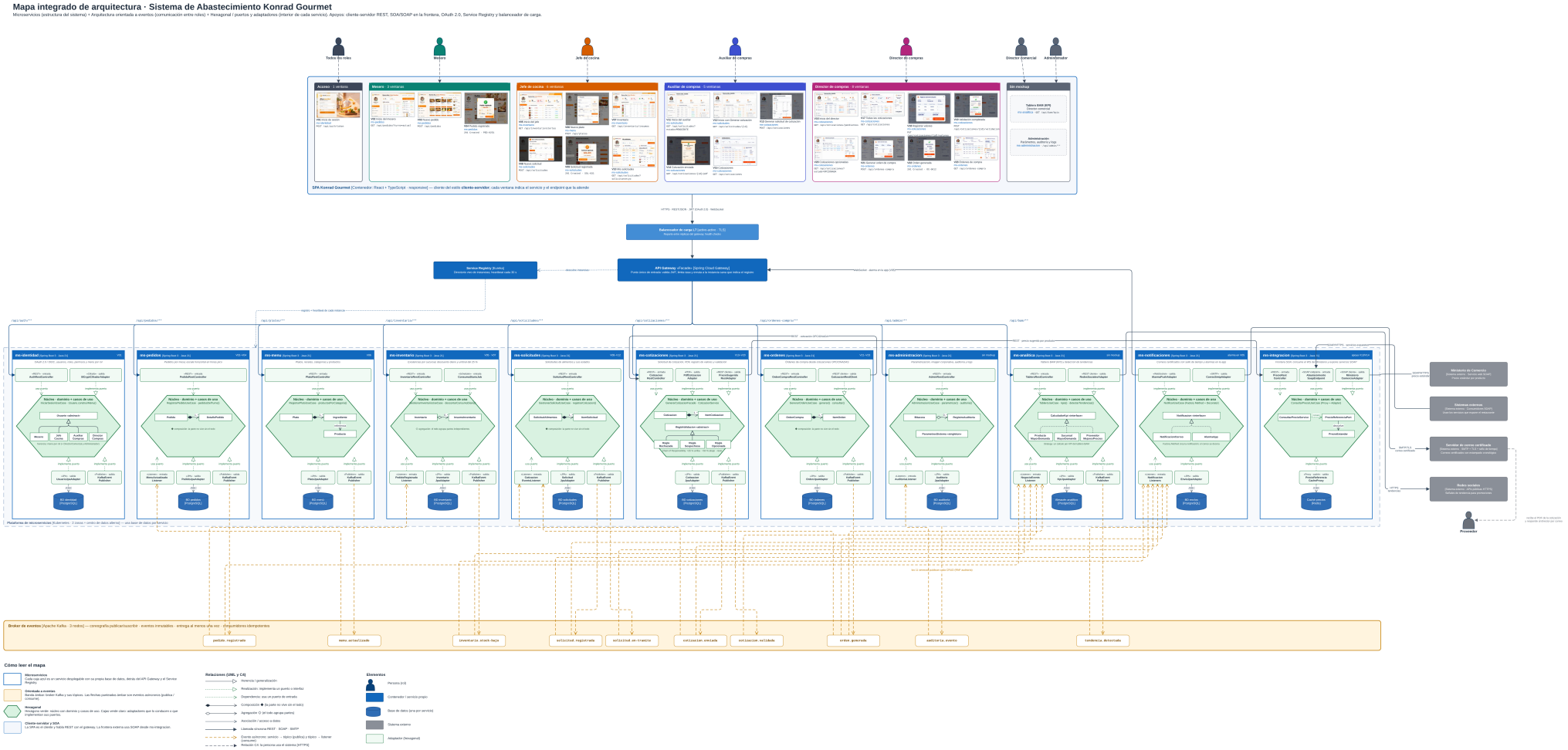
9 El mapa integrado y su notación

La página 1 del archivo .drawio integra las tres arquitecturas en una sola vista que se lee de arriba abajo. Arriba están las personas del C4, una por rol y con el color del rol. Debajo, el contenedor SPA agrupa las 23 ventanas por rol y cada miniatura indica qué servicio y qué endpoint la atienden. Las peticiones bajan por HTTPS al balanceador y al API Gateway, que se apoya en el Service Registry. La fila central son los once microservicios: cada caja azul contiene arriba sus adaptadores de entrada y de salida síncrona, al centro el hexágono con el dominio y abajo los listeners, el adaptador JPA con su base y el publicador de eventos. La banda ámbar del fondo es Kafka con sus diez tópicos. A la derecha están los sistemas externos y el proveedor.

La notación combina UML (para las relaciones entre clases) y C4 (para personas, contenedores y sistemas). Las mismas formas y colores se usan en todas las páginas del archivo.

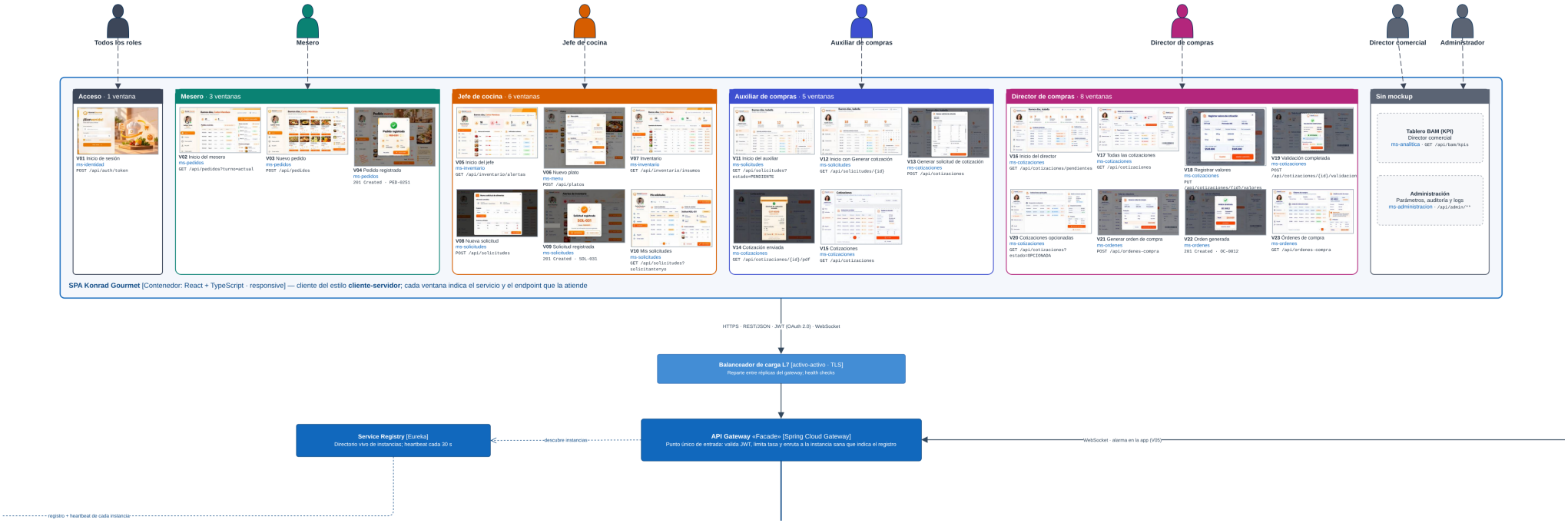
Símbolo	Significado	Ejemplo en el diagrama
	Herencia o generalización: la subclase es un tipo de la superclase	Mesero, JefeCocina, AuxiliarCompras y DirectorCompras heredan de Usuario; ReglaRechazada hereda de ReglaValidacion
	Realización: una clase implementa una interfaz o un puerto	UsuarioJpaAdapter implementa UsuarioRepositoryPort; flechas «implementa puerto» del mapa
	Dependencia: un elemento usa a otro sin guardarlo	ConsultarPrecioService «usa» PrecioReferenciaPort; flechas «usa puerto» de los adaptadores de entrada
	Composición (rombo lleno): la parte no existe sin el todo	Pedido ◆ DetallePedido 1..*; Cotizacion ◆ ItemCotizacion 1..*
	Agregación (rombo vacío): el todo agrupa partes que existen por sí solas	Inventario ◇ InsumoInventario; Bitacora ◇ RegistroAuditoria
	Asociación navegable, con rol y multiplicidad	TableroService → CalculadorKpi (estrategias 1..*)
	Llamada síncrona, con su protocolo en la etiqueta	SPA → balanceador [HTTPS]; ms-integracion → Ministerio [SOAP/HTTPS]
	Evento asíncrono que se publica o se consume	ms-cotizaciones → cotizacion.enviada → ms-notificaciones
	Relación C4 entre una persona o sistema externo y el sistema	Mesero → SPA «Registra pedidos por mesa»
	Contenedor C4: microservicio desplegable	ms-pedidos [Spring Boot 3 · Java 21]
	Núcleo hexagonal: dominio y casos de uso	Uno por microservicio
	Base de datos o caché	BD pedidos [PostgreSQL]; caché de precios [Redis]
	Sistema externo	Ministerio de Comercio, servidor de correo certificado, redes sociales

Figura 1 · Mapa integrado de las tres arquitecturas



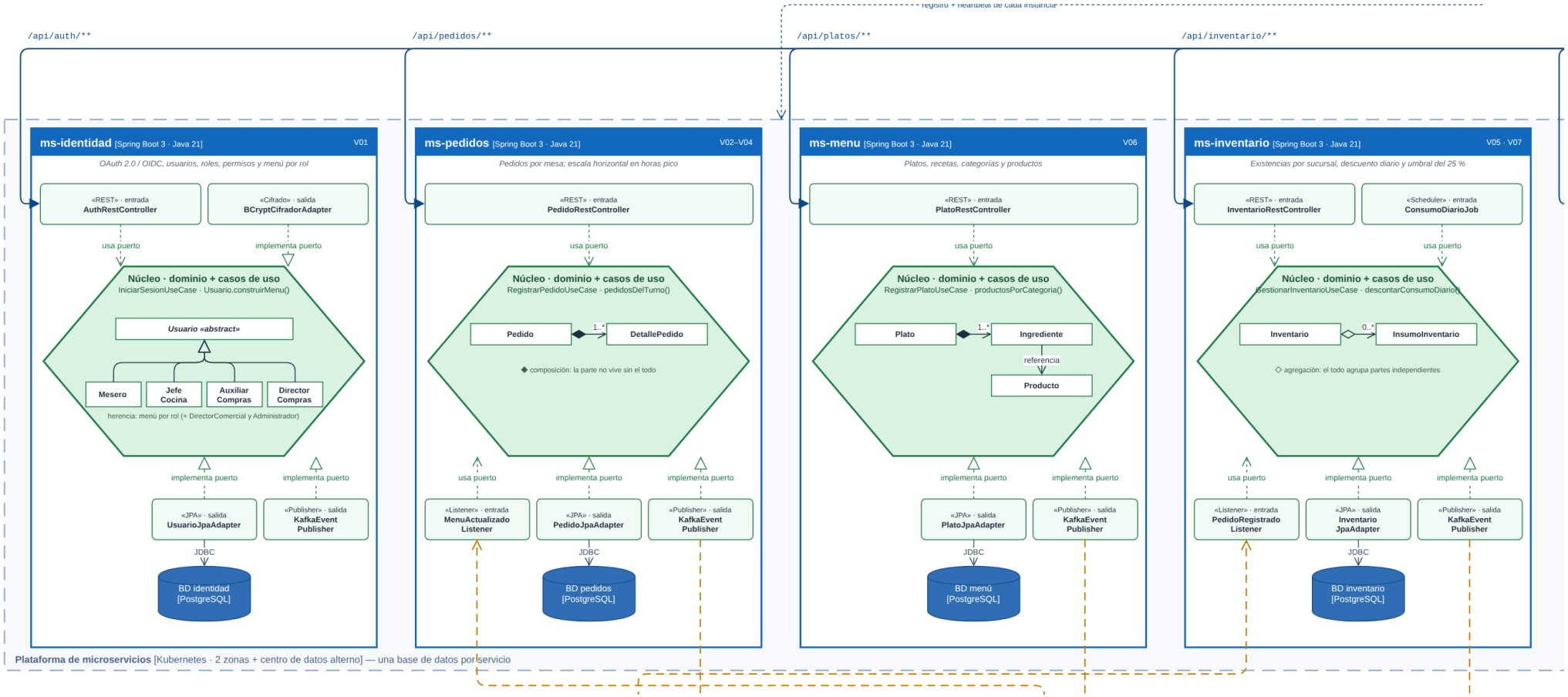
Página 1 del .drawio. De arriba abajo: personas por rol, SPA con las 23 ventanas, balanceador, API Gateway y Service Registry, los once microservicios con su núcleo hexagonal, el broker de eventos y, a la derecha, los sistemas externos. Amplíe en el visor para leer los detalles; las figuras 2 a 6 muestran cada zona de cerca.

Figura 2 · Clientes, ventanas y punto de entrada



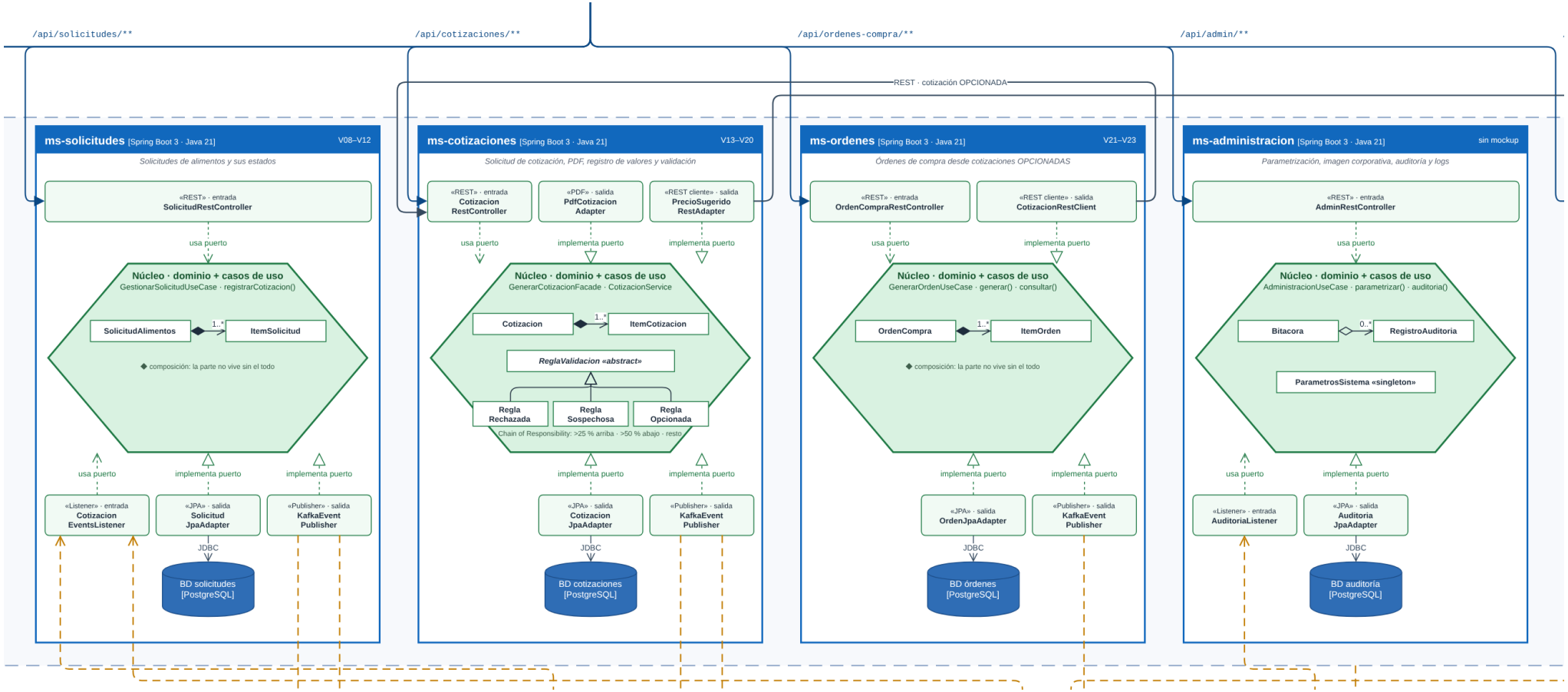
Las ventanas se agrupan por rol con el color de su actor y cada miniatura indica el servicio y el endpoint que la atiende. El director comercial y el administrador no tienen mockup, pero ya tienen servicio y ruta. Todas las peticiones bajan por HTTPS al balanceador y al API Gateway «Facade», que descubre instancias en el Service Registry.

Figura 3 · ms-identidad, ms-pedidos, ms-menu y ms-inventario



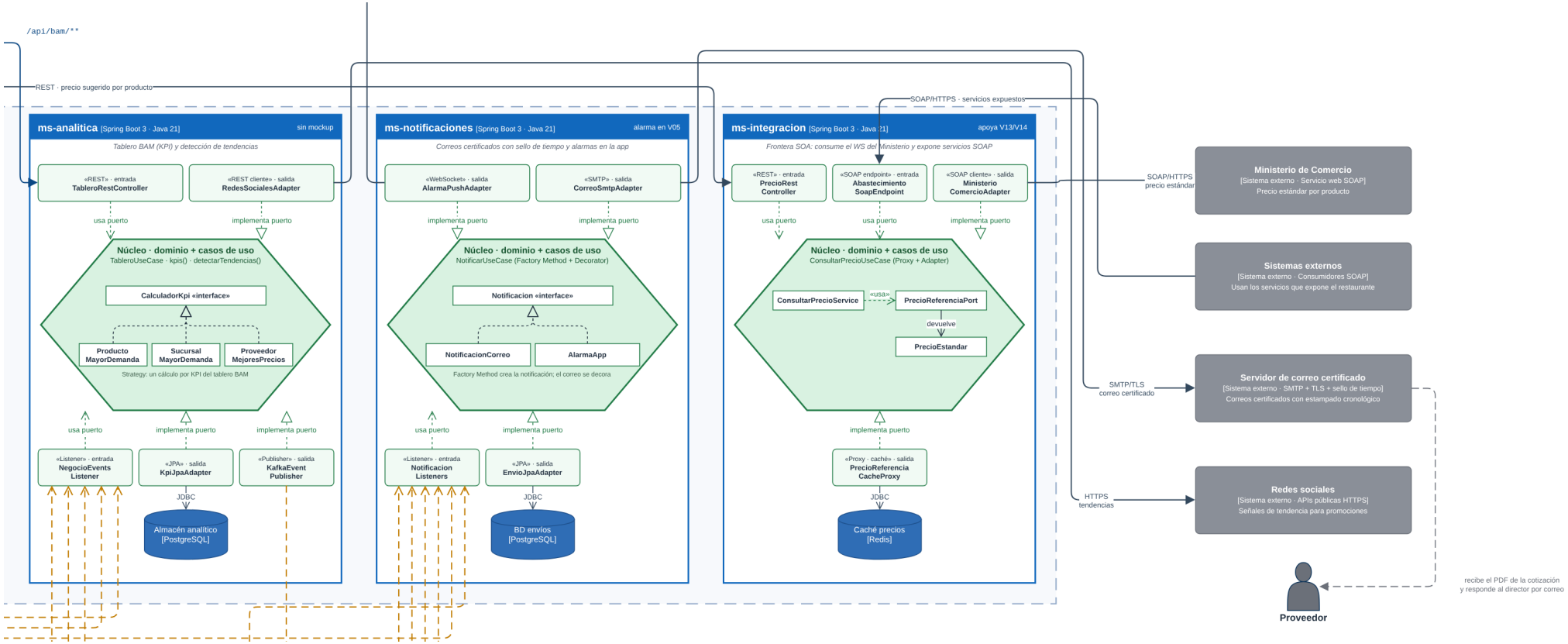
Encima de cada hexágono, los adaptadores de entrada (REST, Scheduler) y de salida síncrona (BCrypt); dentro, el dominio con su relación UML principal (herencia de Usuario, composición Pedido–DetallePedido, agregación Inventario–InsumoInventario); debajo, listeners de eventos, adaptador JPA con su base propia y publicador Kafka.

Figura 4 · ms-solicitudes, ms-cotizaciones, ms-ordenes y ms-administracion



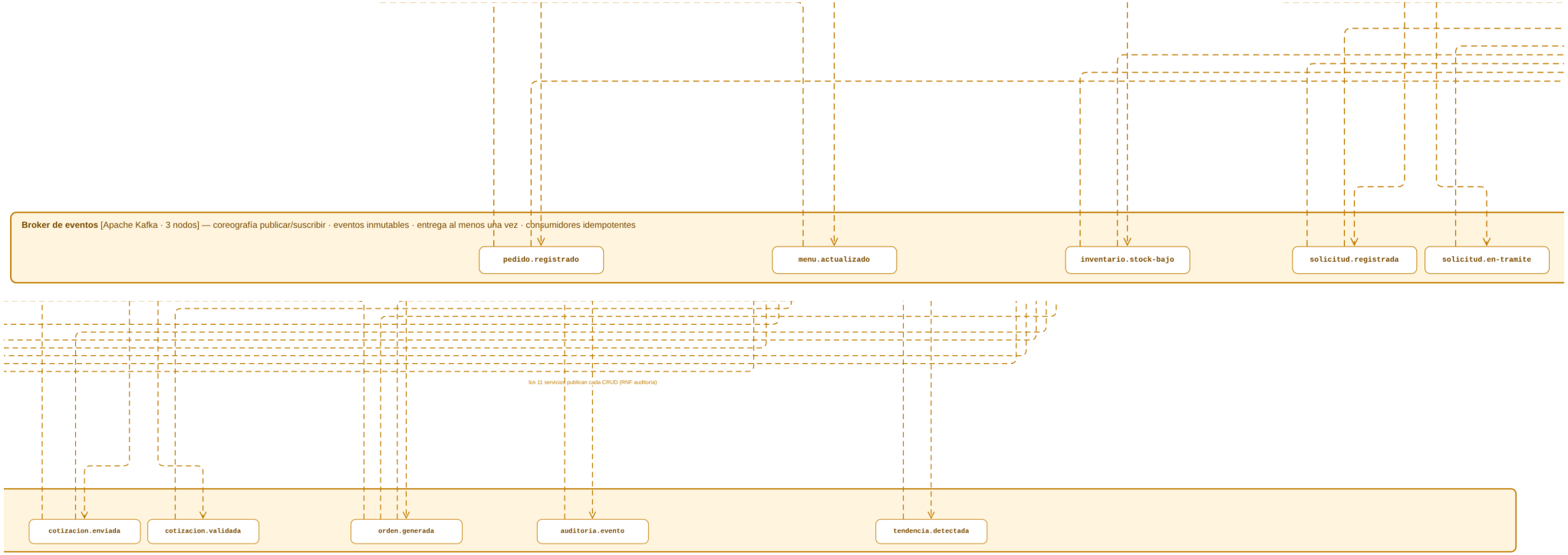
En ms-cotizaciones se ve la cadena de reglas de validación (Chain of Responsibility) y los adaptadores de PDF y de precio sugerido; en ms-administracion, la agregación de la bitácora de auditoría y el Singleton de parámetros.

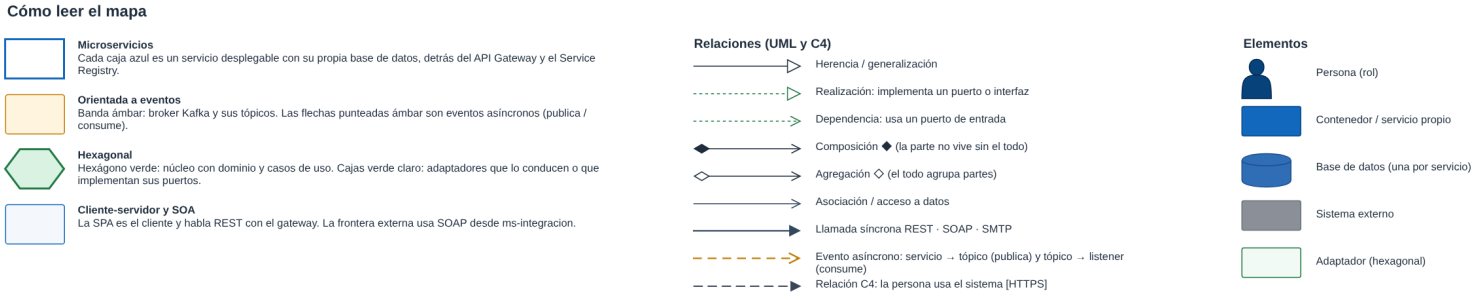
Figura 5 · ms-analitica, ms-notificaciones, ms-integracion y sistemas externos



ms-analitica aplica Strategy para cada KPI; ms-notificaciones combina Factory Method y Decorator para el correo certificado; ms-integracion es la frontera SOA, con el endpoint SOAP hacia afuera y el Proxy con caché delante del adaptador del Ministerio.

Figura 6 · Broker de eventos y leyenda del mapa





Arriba, las flechas punteadas ámbar bajan desde los publicadores hasta cada tópico de Kafka y suben desde el tópico hasta los listeners que lo consumen. Abajo, la leyenda incluida en el propio archivo: arquitecturas, relaciones UML y C4 y elementos.

10 Arquitectura por flujo de ventanas

Las tres arquitecturas están presentes en todo el sistema, pero en cada flujo una de ellas pesa más. La tabla indica cuál y por qué, a partir de lo que muestran las ventanas.

Flujo	Ventanas	Servicios	Domina	Por qué
Acceso	V01	ms-identidad	**Microservicios** + OAuth 2.0	Todos los roles entran por aquí. Separarlo permite escalarlo en los picos de inicio de turno y que una falla de otro servicio no impida autenticarse. El menú lateral de cada rol viaja en el token.
Mesero · pedidos	V02–V04	ms-pedidos (catálogo por evento de ms-menu)	**Microservicios** + **eventos**	Es el flujo de mayor volumen. La respuesta es síncrona e inmediata (PED-0251) y el descuento de inventario y los KPI ocurren después, con pedido.registrado.
Jefe · platos	V06	ms-menu	**Hexagonal** + **eventos**	CRUD con reglas de receta (categoría, producto, cantidad, unidad). menu.actualizado refresca el catálogo del mesero sin acoplar servicios.
Jefe · inventario y alarmas	V05 · V07	ms-inventario, ms-notificaciones	**Eventos**	La tarea diaria descuenta el consumo, evalúa el umbral del 25 % y publica inventario.stock-bajo, que se convierte en alarma en la app (WebSocket) y en correo.
Jefe · solicitudes	V08–V10	ms-solicitudes	**Hexagonal** + **eventos**	La validación de formato vive en el dominio. El paso de PENDIENTE a EN TRÁMITE no lo hace el jefe sino un evento de ms-cotizaciones.
Auxiliar · cotizaciones	V11–V15	ms-solicitudes, ms-cotizaciones, ms-integracion, ms-notificaciones	**Eventos** + **Hexagonal** + SOA	Un clic (Facade) dispara precio del Ministerio por SOAP, PDF, correo al proveedor y cambio de estado de la solicitud, sin bloquear al auxiliar.
Director · validación	V16–V20	ms-cotizaciones	**Hexagonal**	La validación es una regla de negocio pura y comprobable: más de 25 % por encima, RECHAZADA; más de 50 % por debajo, SOSPECHOSA; el resto, OPCIONADA (Chain of Responsibility).
Director · órdenes	V21–V23	ms-ordenes	**Microservicios** + REST	El director espera el número de orden, así que la consulta a ms-cotizaciones es síncrona; la propagación (solicitud, proveedor, KPI) es asíncrona con orden.generada.
Director comercial · BAM	sin mockup	ms-analitica	**Eventos** + Strategy	Los KPI se calculan sobre un almacén propio alimentado por eventos, sin consultar las bases de otros servicios.
Administrador	sin mockup	ms-administracion	**Eventos** + Singleton	La auditoría de cada CRUD llega por auditoria.evento; los parámetros y la imagen corporativa viven en ParametrosSistema.
Sistemas externos	—	ms-integracion	SOA + Proxy y Adapter	Cumple el requerimiento de integración y aísla al resto del sistema de SOAP.

Recorrido completo de una compra

El flujo más largo del sistema, de la solicitud del jefe a la orden de compra, muestra cómo se alternan las llamadas síncronas (el usuario espera) y los eventos (el sistema reacciona por su cuenta).

#	Ventana	Qué pasa	Tipo	Mensaje
1	V08 → V09	El jefe registra la solicitud de alimentos	**Síncrono**	`POST /api/solicitudes → 201 SOL-031 PENDIENTE`
2	—	Confirmación al jefe por correo	**Evento**	`solicitud.registrada → ms-notificaciones`
3	V11 → V12	El auxiliar filtra solicitudes y abre el detalle	**Síncrono**	`GET /api/solicitudes · GET /api/solicitudes/{id}`

#	Ventana	Qué pasa	Tipo	Mensaje
4	V13 → V14	El auxiliar genera la solicitud de cotización	Síncrono	`POST /api/cotizaciones → precio del Ministerio (REST → SOAP), PDF, ENVIADA`
5	—	PDF al proveedor y revisión de la solicitud	Evento	`cotizacion.enviada → ms-notificaciones y ms-solicitudes`
6	V10	La solicitud del jefe pasa a EN TRÁMITE	Evento	`solicitud.en-tramite → aviso al jefe`
7	—	El proveedor responde por correo	Fuera del sistema	—
8	V18	El director registra los valores del proveedor	Síncrono	`PUT /api/cotizaciones/{id}/valores`
9	V19	El director valida la cotización	Síncrono	`POST /api/cotizaciones/{id}/validacion → cadena de reglas → OPCIONADA`
10	—	KPI de proveedor con mejores precios	Evento	`cotizacion.validada → ms-analitica`
11	V20 → V22	El director elige la opcionada y genera la orden	Síncrono	`POST /api/ordenes-compra (consulta REST a ms-cotizaciones) → 201 OC-0012`
12	—	La orden se propaga	Evento	`orden.generada → ms-solicitudes, ms-notificaciones, ms-analitica`
13	V23	Seguimiento de órdenes	Síncrono	`GET /api/ordenes-compra`
14	todas	Auditoría de cada CRUD	Evento	`auditoria.evento → ms-administracion`

11 Eventos del sistema

Los diez tópicos del broker salen de las costuras entre roles que se identificaron en las ventanas. Cada uno tiene un único productor (salvo la auditoría, que publican todos) y uno o más consumidores.

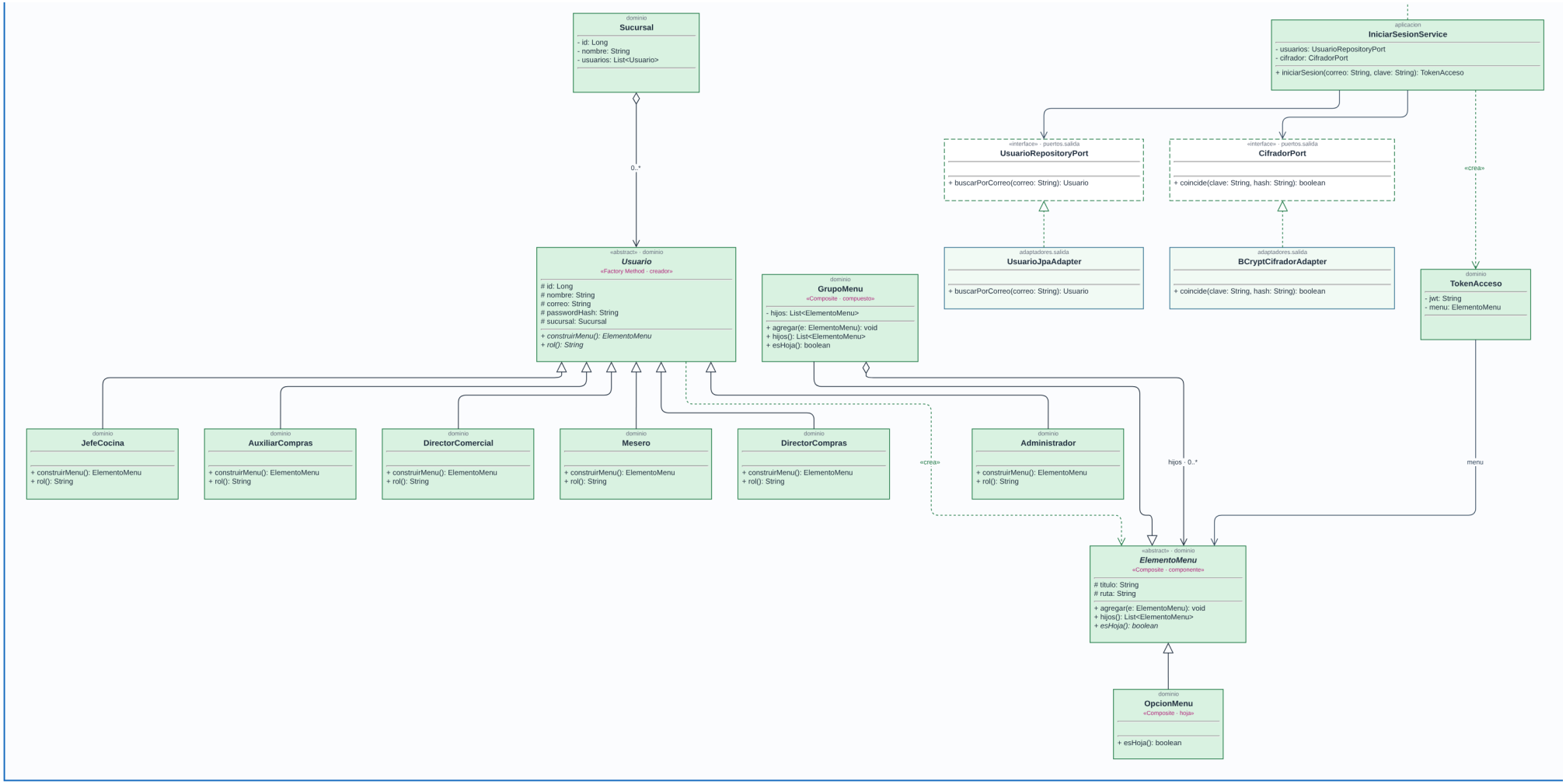
Tópico	Publica	Consumen	Qué lo dispara y para qué
`menu.actualizado`	ms-menu	ms-pedidos	Plato creado o editado (06): actualiza el catálogo del mesero (03)
`pedido.registrado`	ms-pedidos	ms-inventario, ms-analitica	Pedido creado (03/04): consumo para el descuento diario y KPI de demanda
`inventario.stock-bajo`	ms-inventario	ms-notificaciones, ms-analitica	Existencia ≤ 25 %: alarma en la app (05) y correo al jefe
`solicitud.registrada`	ms-solicitudes	ms-notificaciones, ms-analitica	Solicitud PENDIENTE (08/09): correo de confirmación y KPI
`solicitud.en-tramite`	ms-solicitudes	ms-notificaciones	Todos los alimentos cotizados: el jefe ve EN TRÁMITE (10)
`cotizacion.enviada`	ms-cotizaciones	ms-solicitudes, ms-notificaciones	Cotización ENVIADA (13/14): PDF por correo al proveedor y revisión de EN TRÁMITE
`cotizacion.validada`	ms-cotizaciones	ms-analitica	RECHAZADA, SOSPECHOSA u OPCIONADA (19): KPI de proveedor con mejores precios
`orden.generada`	ms-ordenes	ms-solicitudes, ms-notificaciones, ms-analitica	Orden de compra (21/22): la solicitud avanza y se avisa al proveedor
`auditoria.evento`	los 11 servicios	ms-administracion	Cada CRUD de los 11 servicios: acción, usuario, fecha y hora
`tendencia.detectada`	ms-analitica	ms-notificaciones	Productos más solicitados o tendencia en redes: correo de promociones a los jefes

12 Patrones de diseño en el nivel de código

El profesor pidió que en el nivel 4 del C4 se vea dónde va cada patrón. En las páginas 4a a 4e del .drawio cada clase con patrón lleva su estereotipo en magenta, y el esqueleto Java implementa el mismo comportamiento. La tabla resume los diez patrones, dónde están y qué costo tienen.

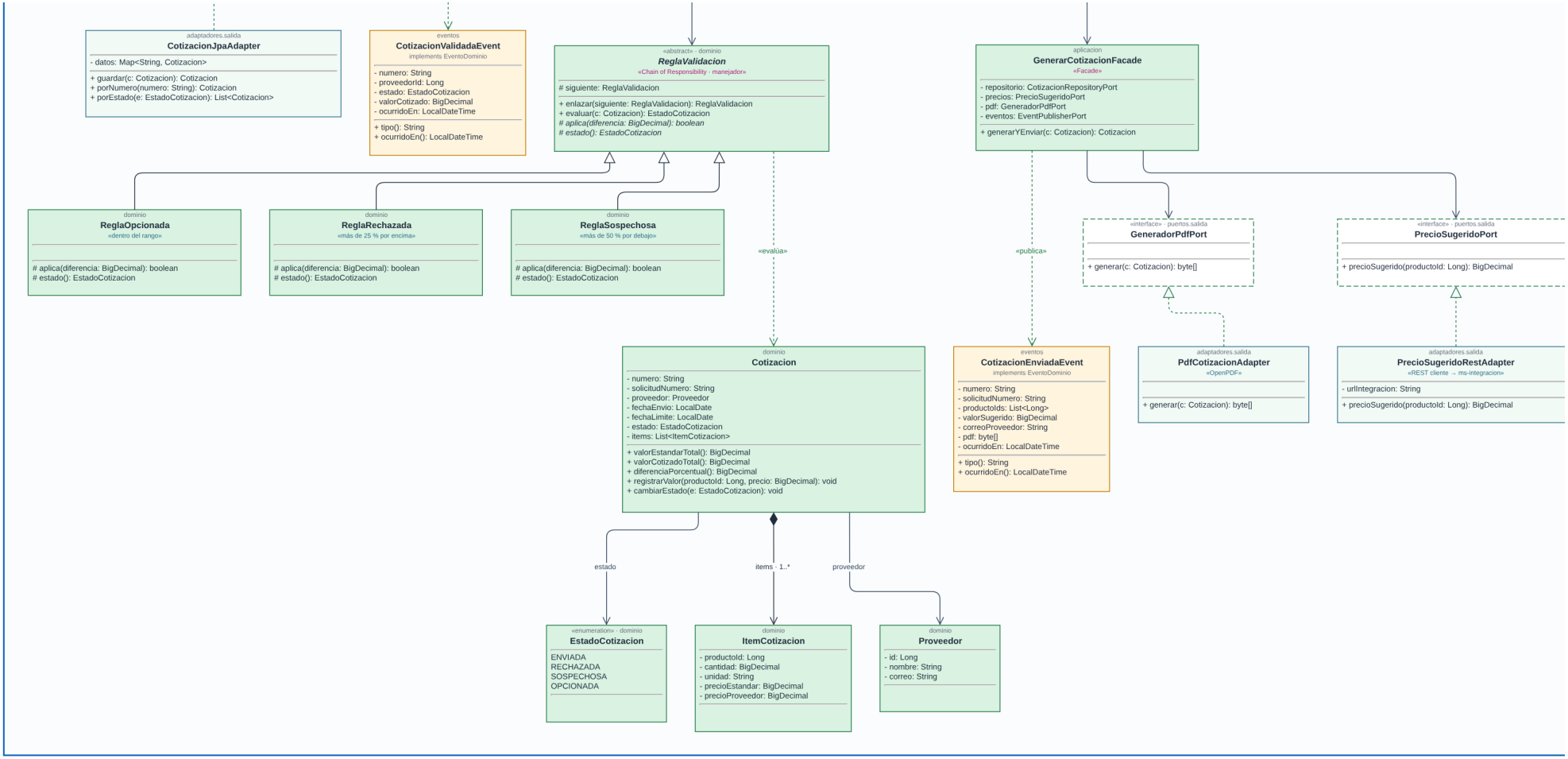
Patrón	Clases	Servicio · ventanas	Por qué aquí	Consecuencia
Facade	GenerarCotizacionFacade; a escala de sistema, el API Gateway	ms-cotizaciones · V13, V14	Un clic coordina precio sugerido, repositorio, PDF y evento	El controlador hace una sola llamada; la fachada solo coordina, no decide
Chain of Responsibility	ReglaValidacion → ReglaRechazada → ReglaSospechosa → ReglaOpcionada	ms-cotizaciones · V19	Tres reglas excluyentes que se evalúan en orden; una regla nueva no toca las demás	El orden importa, así que la cadena se arma en un solo lugar (cadenaPorDefecto)
Composite	ElementoMenu, GrupoMenu, OpcionMenu	ms-identidad · panel izquierdo de las 23	El menú lateral es un árbol de grupos y opciones	Un rol nuevo no exige cambiar la SPA, que solo recorre el árbol
Factory Method	Usuario.construirMenu() en cada rol; CreadorNotificacion → CreadorCorreo, CreadorAlarmaApp	ms-identidad, ms-notificaciones · V01, V05	La subclase decide qué producto crear: el menú del rol o el tipo de notificación	Una clase creadora por variante
Decorator	CorreoDecorator, CorreoCertificadoDecorator, EstampaCronologicaDecorator sobre CorreoSmtpAdapter	ms-notificaciones · correos de V09, V14, V22	Certificado y sello de tiempo se suman al correo sin modificar el envío SMTP	El orden de los decoradores se fija en la configuración
Adapter	MinisterioComercioAdapter y todos los *Adapter de la hexagonal	ms-integracion · V13	Traduce el servicio SOAP del Ministerio al puerto PrecioReferenciaPort	Un adaptador por cada tecnología externa
Proxy	PrecioReferenciaCacheProxy	ms-integracion · V13	Evita llamar al servicio web por cada producto de cada cotización	Un precio puede quedar desactualizado hasta que vence la caché
Observer	EventPublisherPort, EventListener y los listeners de Kafka	compartido y todos los servicios	Publicar hechos sin conocer a los suscriptores	Consistencia eventual entre servicios
Singleton	ParametrosSistema	ms-administracion · imagen corporativa	Una sola fuente de parámetros por instancia: umbral del 25 %, tema, logos	Estado global: de solo lectura para el resto, recargado cuando cambia
Strategy	CalculadorKpi → ProductoMayorDemanda, SucursalMayorDemanda, ProveedorMejoresPrecios	ms-analitica · tablero BAM	Cada KPI es un algoritmo intercambiable sobre los mismos hechos	Un KPI nuevo es una clase nueva

Figura 7 · Composite y Factory Method en ms-identidad



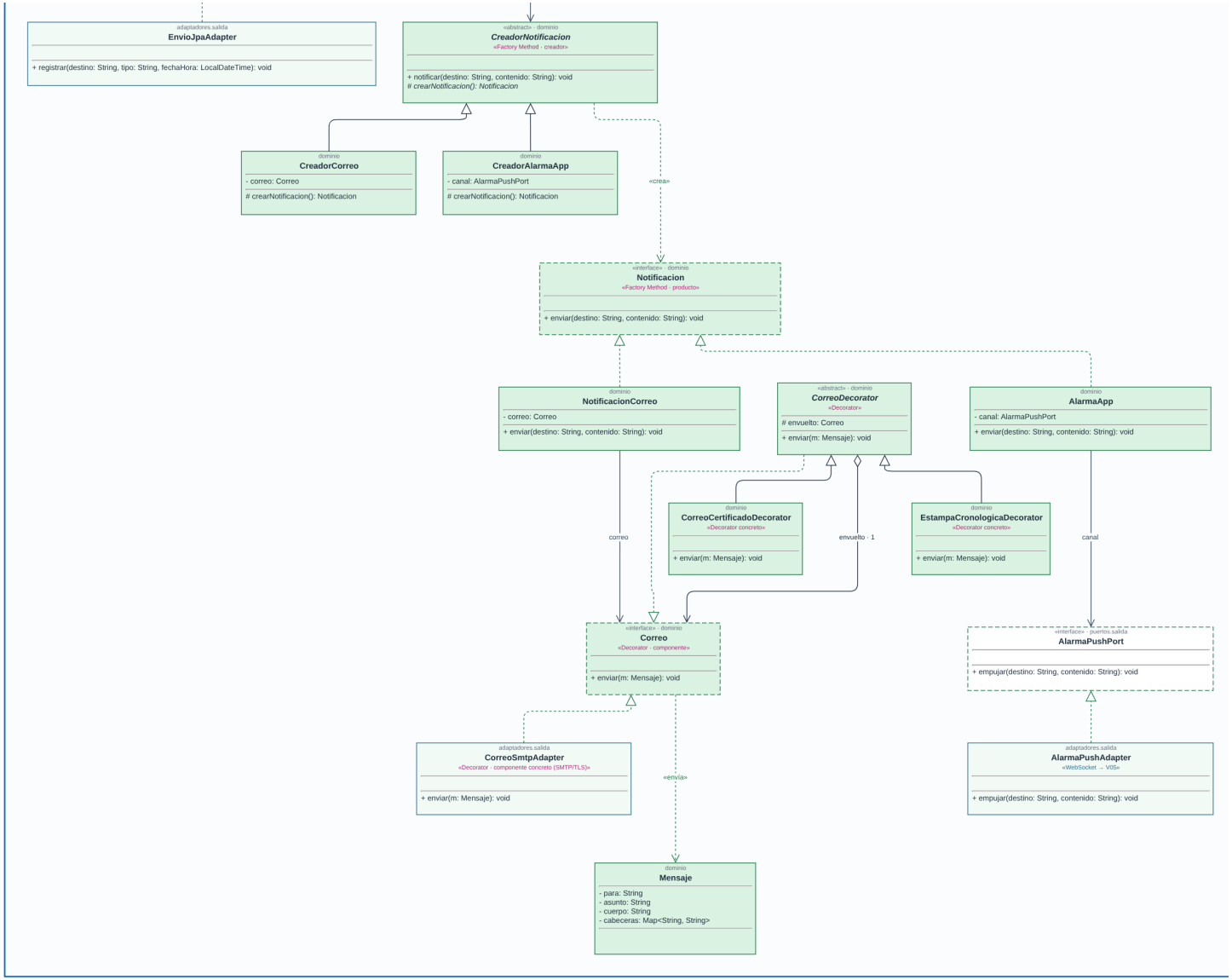
Fragmento de la página 4a. Usuario «abstract» es el creador del Factory Method: cada rol implementa construirMenu() y devuelve un árbol ElementoMenu (Composite) con GrupoMenu y OpcionMenu. A la derecha, IniciarSesionService usa los puertos UsuarioRepositoryPort y CifradorPort, que implementan UsuarioJpaAdapter y BCryptCifradorAdapter (líneas punteadas de realización).

Figura 8 · Chain of Responsibility y Facade en ms-cotizaciones



Fragmento de la página 4c. ReglaValidacion enlaza ReglaRechazada, ReglaSospechosa y ReglaOpcionada (herencia con triángulo hueco). GenerarCotizacionFacade depende de los puertos de repositorio, precio sugerido y PDF. Cotizacion compone sus ItemCotizacion (rombo lleno) y se asocia al Proveedor y a su EstadoCotizacion.

Figura 9 · Factory Method y Decorator en ms-notificaciones



Fragmento de la página 4d. CreadorNotificacion «abstract» delega en CreadorCorreo y CreadorAlarmaApp la creación de la Notificacion. CorreoDecorator agrega (rombo vacío) un Correo y sus subclases añaden el certificado y el sello de tiempo; CorreoSmtpAdapter es el componente concreto que habla SMTP con TLS.

13 Del diagrama al código

Un solo modelo, tres salidas

El mapa, el diagrama de clases, el esqueleto Java y las tablas de este documento se generan desde el mismo modelo, de modo que los nombres coinciden uno a uno. Antes de entregar se comprobó automáticamente que cada clase que aparece en el mapa (adaptadores, núcleo de dominio y casos de uso) existe en el código y que el método asignado a cada una de las 23 ventanas está declarado en su controlador.

Estructura del esqueleto

El código está en el paquete co.edu.konradlorenz.<servicio>.<capa>, con doce paquetes de servicio (los once microservicios más compartido) y 155 clases, además de la clase DemoArquitectura. Está escrito en Java 21 sin dependencias externas para que compile solo con javac y se pueda revisar en cualquier equipo. En el proyecto real, cada paquete de servicio se convierte en un proyecto Spring Boot y los adaptadores reciben sus anotaciones (@RestController, @Repository, @KafkaListener) sin tocar el dominio, que es precisamente lo que promete la arquitectura hexagonal. Los métodos marcados en el diagrama con ‹Vxx› llevan en el código el comentario «Atiende la ventana Vxx de los mockups».

Comprobación con la demostración

DemoArquitectura ejecuta los patrones sin base de datos ni broker, usando adaptadores en memoria. Su salida coincide con los datos de los mockups:

Patrón	Salida de la demostración	Ventana
Composite + Factory Method	Menú DIRECTOR_COMPRAS con Inicio, Cotizaciones (Todas, Pendientes de validar, Opcionadas), Órdenes de compra, Mi perfil y Cerrar sesión	Panel izquierdo de V16–V23
Hexagonal + Observer	Pedido registrado, evento pedido.registrado publicado, total $82.000	V03 y V04
Facade + Chain of Responsibility	COT-018: estándar $520.000, proveedor $545.000, diferencia 4,81 %, OPCIONADA. COT-019: +34,62 %, RECHAZADA. COT-020: -61,54 %, SOSPECHOSA	V13, V18 y V19
Factory Method + Decorator	Correo a jefe.centro@konradgourmet.com con encabezados X-Certificado y X-Sello-Tiempo; alarma «Res al 23 %: realice una solicitud»	V05 y correos de V09
Proxy + Adapter	Primer precio de la res desde el servicio web; el segundo, desde la caché del Proxy	V13
Singleton	umbral.stock = 25 %, misma instancia en dos llamadas	Parametrización
Strategy	Producto con mayor demanda: Bandeja paisa (40); proveedor con mejores precios: Proveedor B	Tablero BAM

Para compilar y ejecutar: `javac -d bin $(find src -name "*.java")` y luego `java -cp bin co.edu.konradlorenz.demo.DemoArquitectura`.

14 Contexto y despliegue

Nivel 1 del C4

El diagrama de contexto muestra el sistema como una sola caja con seis tipos de persona y cuatro sistemas externos: el servicio web SOAP del Ministerio de Comercio, el servidor de correo certificado, las APIs de redes sociales y los sistemas externos que consumen los servicios SOAP del restaurante. El proveedor aparece como persona externa porque recibe el PDF por correo y responde fuera del sistema; por eso el director digita sus valores en la ventana 18.

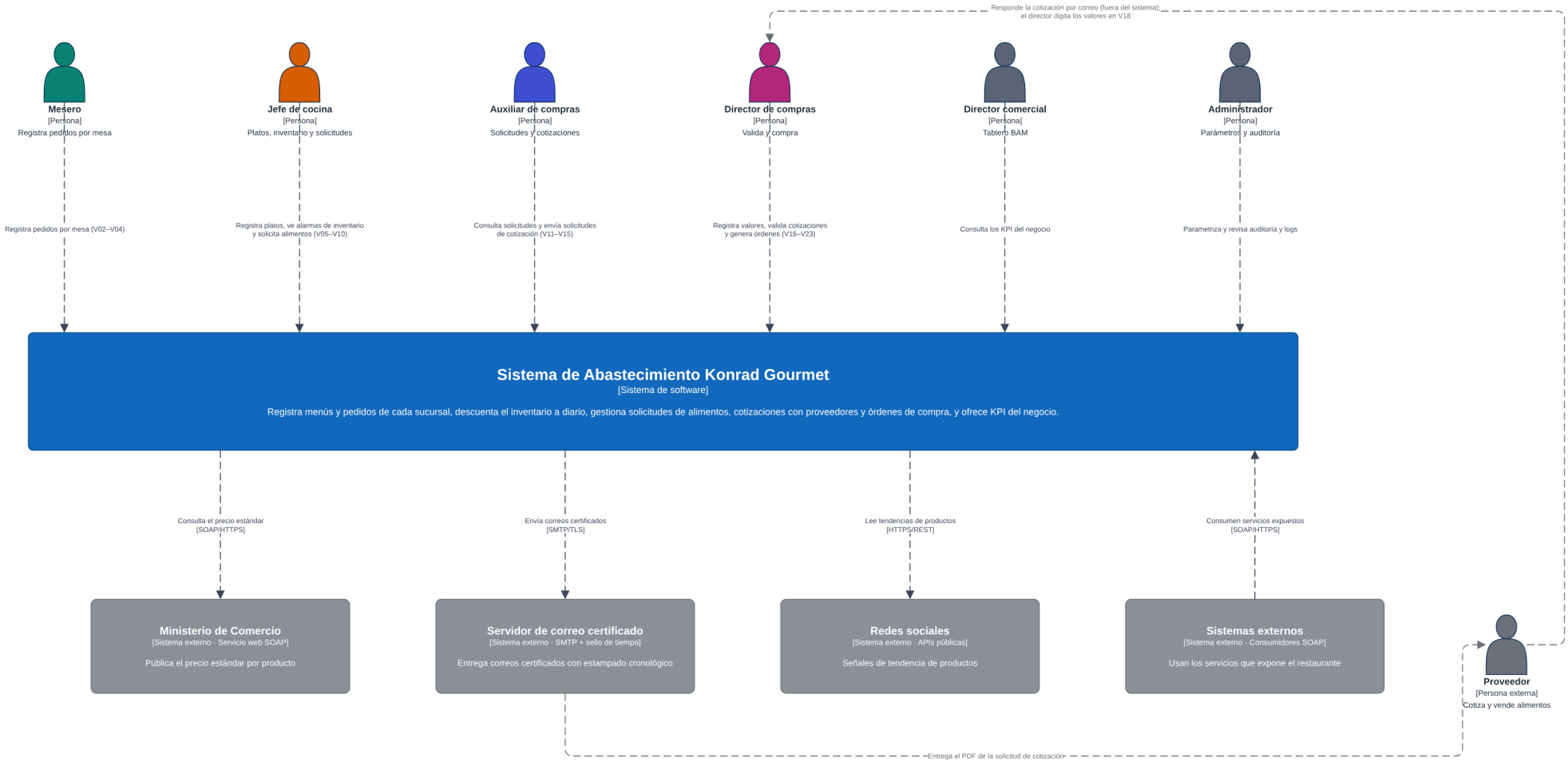
Despliegue para el 99,7 %

Un 99,7 % de disponibilidad permite unas 26,3 horas de caída al año, poco más de dos horas al mes. El diagrama de despliegue lo cubre con redundancia en cada capa: DNS global con conmutación, CDN con WAF, balanceador activo-activo, tres réplicas del gateway, dos del Service Registry, réplicas mínimas y máximas por servicio con autoescalado al 65 % de CPU en un clúster Kubernetes repartido en tres zonas, Kafka con tres brokers y PostgreSQL con réplica síncrona. El centro de datos alterno, en otra región, mantiene las mismas imágenes con réplicas mínimas encendidas, recibe los tópicos con MirrorMaker 2 y las bases por réplica asíncrona; los objetivos son perder como máximo cinco minutos de datos y recuperar el servicio en una hora. El respaldo diario combina copia nocturna y registros continuos con retención de 30 días.

Figura 10 · Nivel 1 del C4, diagrama de contexto

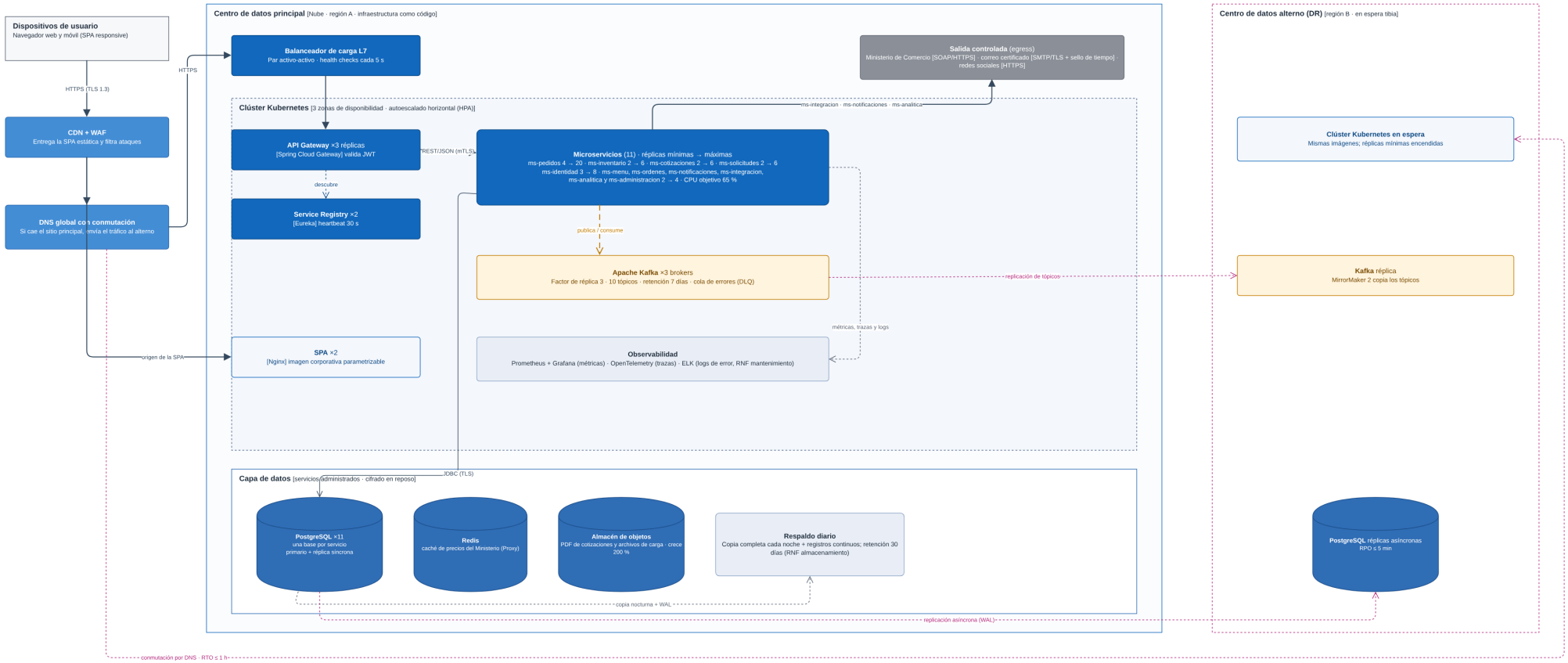
Nivel 1 (C4) · Diagrama de contexto

El sistema como una caja: quién lo usa y con qué sistemas externos se integra. Las relaciones son flechas punteadas con su propósito y protocolo.



Página 2 del .drawio. Las relaciones son flechas punteadas con su propósito y su protocolo; las personas llevan el color de su rol en el mapa.

Figura 11 · Diagrama de despliegue



Página 5 del .drawio. Centro de datos principal con el clúster Kubernetes, la capa de datos y la salida controlada hacia los sistemas externos; a la derecha, el centro de datos alterno. Las líneas magenta son la replicación y la conmutación ante desastres.

15 Trazabilidad de las 23 ventanas

Cada ventana del prototipo está asignada a un servicio, a un método de su controlador REST y a un endpoint. La columna de servicio es la misma que aparece bajo cada miniatura en el mapa; el método existe en el esqueleto Java con el comentario de la ventana que atiende.

V	Rol	Ventana	Servicio	Controlador y método	Endpoint
01	Acceso	Inicio de sesión	ms-identidad	`AuthRestController.iniciarSesion()`	`POST /api/auth/token`
02	Mesero	Inicio del mesero	ms-pedidos	`PedidoRestController.pedidosDelTurno()`	`GET /api/pedidos?turno=actual`
03	Mesero	Nuevo pedido	ms-pedidos	`PedidoRestController.registrar()`	`POST /api/pedidos`
04	Mesero	Pedido registrado	ms-pedidos	`PedidoRestController.registrar() → 201`	`201 Created · PED-0251`
05	Jefe de cocina	Inicio del jefe	ms-inventario	`InventarioRestController.alertas()`	`GET /api/inventario/alertas`
06	Jefe de cocina	Nuevo plato	ms-menu	`PlatoRestController.crear()`	`POST /api/platos`
07	Jefe de cocina	Inventario	ms-inventario	`InventarioRestController.insumos()`	`GET /api/inventario/insumos`
08	Jefe de cocina	Nueva solicitud	ms-solicitudes	`SolicitudRestController.registrar()`	`POST /api/solicitudes`
09	Jefe de cocina	Solicitud registrada	ms-solicitudes	`SolicitudRestController.registrar() → 201`	`201 Created · SOL-031`
10	Jefe de cocina	Mis solicitudes	ms-solicitudes	`SolicitudRestController.misSolicitudes()`	`GET /api/solicitudes ?solicitante=yo`
11	Auxiliar de compras	Inicio del auxiliar	ms-solicitudes	`SolicitudRestController.consultar()`	`GET /api/solicitudes ?estado=PENDIENTE`
12	Auxiliar de compras	Inicio con Generar cotización	ms-solicitudes	`SolicitudRestController.detalle()`	`GET /api/solicitudes/{id}`
13	Auxiliar de compras	Generar solicitud de cotización	ms-cotizaciones	`CotizacionRestController.generar()`	`POST /api/cotizaciones`
14	Auxiliar de compras	Cotización enviada	ms-cotizaciones	`CotizacionRestController.pdf()`	`GET /api/cotizaciones/{id}/pdf`
15	Auxiliar de compras	Cotizaciones	ms-cotizaciones	`CotizacionRestController.consultar()`	`GET /api/cotizaciones`
16	Director de compras	Inicio del director	ms-cotizaciones	`CotizacionRestController.pendientesDeAtencion()`	`GET /api/cotizaciones /pendientes`
17	Director de compras	Todas las cotizaciones	ms-cotizaciones	`CotizacionRestController.consultar()`	`GET /api/cotizaciones`
18	Director de compras	Registrar valores	ms-cotizaciones	`CotizacionRestController.registrarValores()`	`PUT /api/cotizaciones/{id} /valores`
19	Director de compras	Validación completada	ms-cotizaciones	`CotizacionRestController.validar()`	`POST /api/cotizaciones/{id} /validacion`
20	Director de compras	Cotizaciones opcionadas	ms-cotizaciones	`CotizacionRestController.opcionadas()`	`GET /api/cotizaciones ?estado=OPCIONADA`
21	Director de compras	Generar orden de compra	ms-ordenes	`OrdenCompraRestController.generar()`	`POST /api/ordenes-compra`
22	Director de compras	Orden generada	ms-ordenes	`OrdenCompraRestController.generar() → 201`	`201 Created · OC-0012`
23	Director de compras	Órdenes de compra	ms-ordenes	`OrdenCompraRestController.consultar()`	`GET /api/ordenes-compra`

16 Riesgos y mitigaciones

Riesgo	Impacto	Mitigación
El jefe ve PENDIENTE unos segundos después de que el auxiliar envió la cotización	Confusión sobre el estado real de la solicitud	Mostrar la fecha de última actualización en la ventana 10 y vigilar que la propagación de eventos tarde menos de unos segundos
El servicio web del Ministerio no responde	No se puede calcular el precio estándar de una cotización	Proxy con caché en Redis; si no hay precio, la cotización queda pendiente de precio y se reintenta sin bloquear al auxiliar
Un servicio guarda en su base pero falla al publicar el evento	Otros servicios no se enteran del cambio	Patrón outbox transaccional: el evento se guarda en la misma transacción y un proceso lo publica después
Operación compleja de once servicios	Errores de despliegue y diagnóstico lento	Kubernetes, infraestructura como código, trazas con identificador de correlación y tableros de observabilidad
Procesos que cruzan servicios (orden generada, solicitud que avanza)	Datos inconsistentes si un paso falla a mitad	Saga coreografiada con eventos y estados explícitos que permiten compensar o reintentar
Costo de once bases de datos	Más infraestructura que un monolito	Servicios administrados; las bases pequeñas pueden compartir servidor con esquemas separados, sin compartir tablas
Seguridad entre servicios	Un servicio comprometido podría llamar a otros	mTLS entre servicios, propagación del JWT y verificación del rol en cada endpoint

17 Supuestos

El enunciado pide declarar y sustentar los supuestos. Estos son los que tomó el diseño:

#	Supuesto	Por qué
S1	Las 1.000 TPS de «compras» corresponden sobre todo a pedidos de los meseros y a operaciones de cotización y orden	Son los flujos transaccionales del enunciado; por eso esos servicios tienen más réplicas
S2	El proveedor responde la cotización por correo, fuera del sistema, y el director digita los valores	Lo dice el enunciado y lo muestra la ventana 18
S3	El precio estándar del Ministerio cambia como mucho una vez al día	Permite la caché del Proxy sin afectar la validación
S4	El descuento de inventario se consolida una vez al día, aunque el consumo se acumula en línea con pedido.registrado	El enunciado pide una verificación diaria; acumular en línea evita recalcular todo al cierre
S5	El director comercial y el administrador no tienen mockup; sus servicios y rutas quedan listos	Sus funciones están en el enunciado pero no en el prototipo entregado
S6	El despliegue es en nube con Kubernetes administrado; el proveedor de nube se decide después	Decidir la tecnología lo más tarde posible, como pidió el profesor
S7	La certificación y el sello de tiempo los presta un servicio externo de correo certificado	El sistema agrega los encabezados y el sello con el Decorator, pero la certificación legal la da un tercero
S8	Las réplicas mínimas y máximas del despliegue son valores iniciales	Deben validarse con pruebas de carga dentro del plan de pruebas

18 Declaración de uso de inteligencia artificial

En la sesión 9 el profesor pidió declarar siempre el uso de inteligencia artificial: la herramienta, el prompt y el mecanismo con el que se generó el resultado. Esta es esa declaración.

Herramienta

Claude, de Anthropic (modelo Claude Opus), usado en la aplicación Claude con herramientas de archivos y ejecución de código. La IA leyó el enunciado del proyecto y las transcripciones de las sesiones de clase, analizó las 23 imágenes de las ventanas aportadas como insumo, propuso la arquitectura y generó por programa el diagrama .drawio (Node.js con el motor de diseño automático ELK), el esqueleto Java y este documento (Python con ReportLab). También verificó que el código compila, que la demostración reproduce los valores de la ventana 19 y que los nombres del diagrama existen en el código.

Prompts utilizados

Se transcriben en orden, con la ortografía corregida:

1. Analiza el PDF, sobre todo las actividades que hacen el jefe de cocina, el mesero, el director de compras y el auxiliar de compras.

2. Según lo anterior analizado, genera el flujo de la web completo con las 15 imágenes obligatoriamente usadas en el flujo (analiza cada imagen, sus botones, su barra lateral y cómo se conectan entre sí), del jefe de cocina, el mesero y el área de compras, y cómo se conectan entre sí: botones, flechas y descripción de lo que hace cada uno. Si hay preguntas, se representan con rombo y el sí y el no. Todas las acciones de cada imagen tienen que tener conexión, aunque sea a otra parte del mismo dashboard (panel izquierdo); deben haber muchísimas conexiones.

3. Ahora añade estas 8 nuevas imágenes al flujo anterior; son del director de compras, de la 16 a la 23.

4. Analiza todo el proyecto anterior, las imágenes de cada ventana, y piensa cuál arquitectura quedaría mejor para cada flujo, mínimo 3 arquitecturas, y de ahí genera un mapa de esas 3 arquitecturas integradas, con sus buenas dimensiones, formas, flechas punteadas y no punteadas, rombos, herencia, etc. Los archivos adjuntos son la explicación del profe de esas arquitecturas para que las analices y de ahí deliberes. Luego genera un reporte en PDF de por qué consideras esas arquitecturas, más el diagrama en draw.io. El diagrama tiene que encajar con las ventanas a mostrar y con el código.

La revisión del contenido, los ajustes y la sustentación ante el cliente corresponden al equipo.

19 Referencias

[1] López, C. A. (2026). Proyecto #1: Implementación sistema de abastecimiento. Arquitectura de Software, Fundación Universitaria Konrad Lorenz.

[2] Sesiones de clase de Arquitectura de Software, sesiones 2 a 10 (2026). Exposiciones de los grupos y explicaciones del profesor, transcripciones.

[3] Brown, S. (2018). The C4 model for visualising software architecture. https://c4model.com

[4] Cockburn, A. (2005). Hexagonal architecture (Ports and Adapters). https://alistair.cockburn.us/hexagonal-architecture/

[5] Newman, S. (2021). Building Microservices (2.ª ed.). O'Reilly Media.

[6] Richardson, C. (2018). Microservices Patterns. Manning.

[7] Gamma et al. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

[8] Fowler, M. (2017). What do you mean by «Event-Driven»? https://martinfowler.com/articles/201701-event-driven.html

[9] Kleppmann, M. (2017). Designing Data-Intensive Applications. O'Reilly Media.

[10] Bass et al. (2021). Software Architecture in Practice (4.ª ed.). Addison-Wesley.

[11] Richards, M. y Ford, N. (2020). Fundamentals of Software Architecture. O'Reilly Media.

[12] Martin, R. C. (2017). Clean Architecture. Prentice Hall.

[13] Hohpe, G. y Woolf, B. (2003). Enterprise Integration Patterns. Addison-Wesley.

[14] Object Management Group (2017). Unified Modeling Language, versión 2.5.1. https://www.omg.org/spec/UML/2.5.1

Anexo A · Nivel de código completo

Las cinco páginas de nivel de código del .drawio, completas. Cada recuadro azul es un microservicio con sus clases agrupadas por capa hexagonal: verde para dominio y aplicación, blanco con borde punteado para puertos (interfaces), verde claro con borde azul para adaptadores y ámbar para eventos. En magenta, el patrón que cumple cada clase.

Figura A1 · Página 4a · compartido e identidad

Nivel 4 (C4) · Código · compartido e identidad

Diagrama de clases UML generado del mismo modelo que el esqueleto Java (paquetes `co.edu.konradlorenz.<servicio>.<capa>`). Los métodos marcados ‹Vxx› atienden esa ventana de los mockups; en magenta, el patrón de diseño que cumple cada clase.

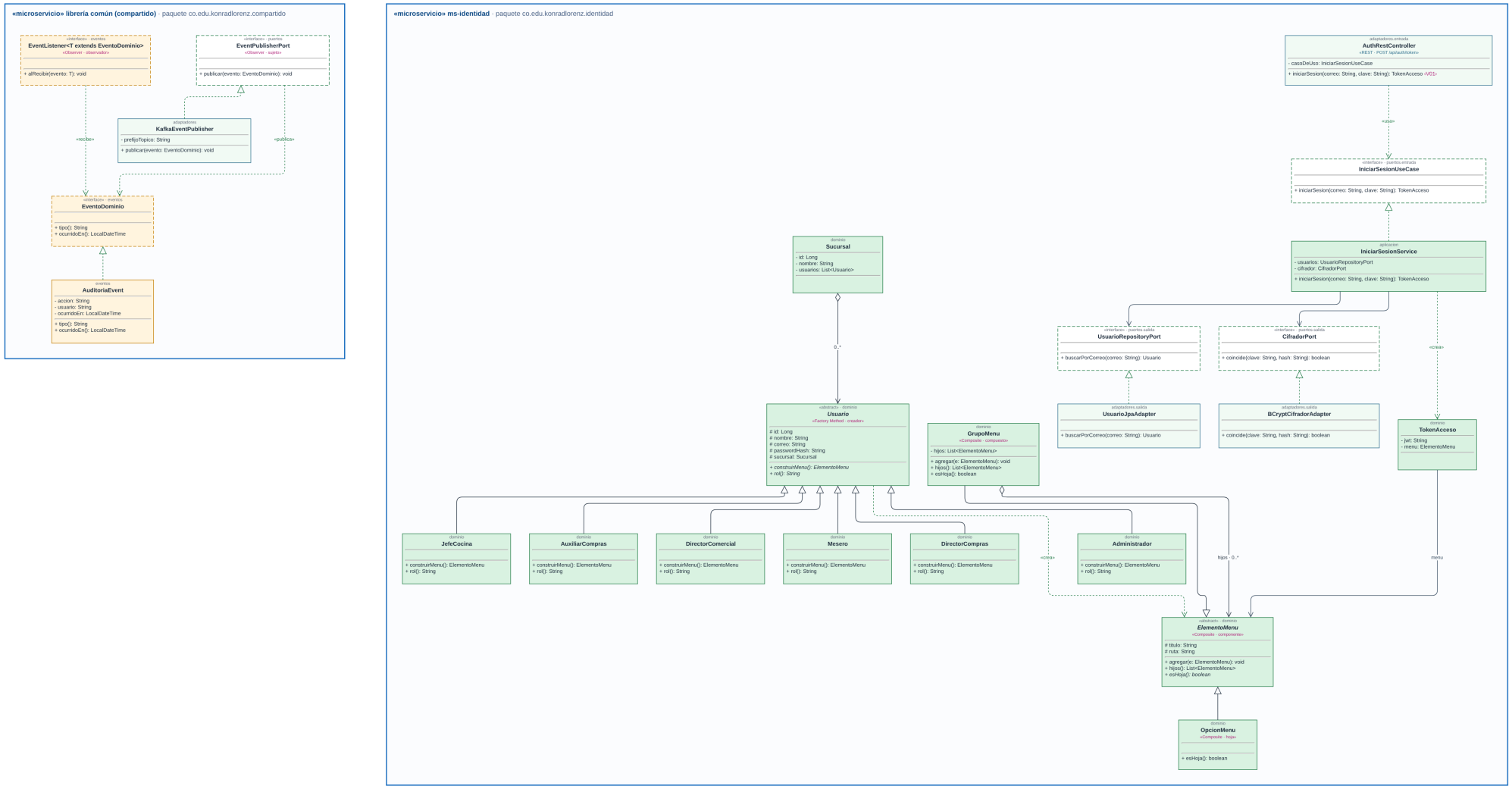
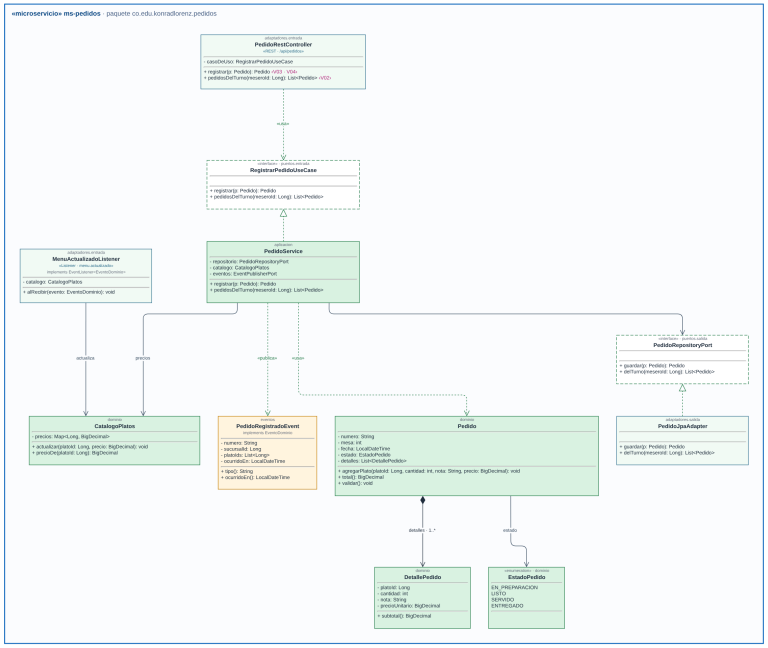
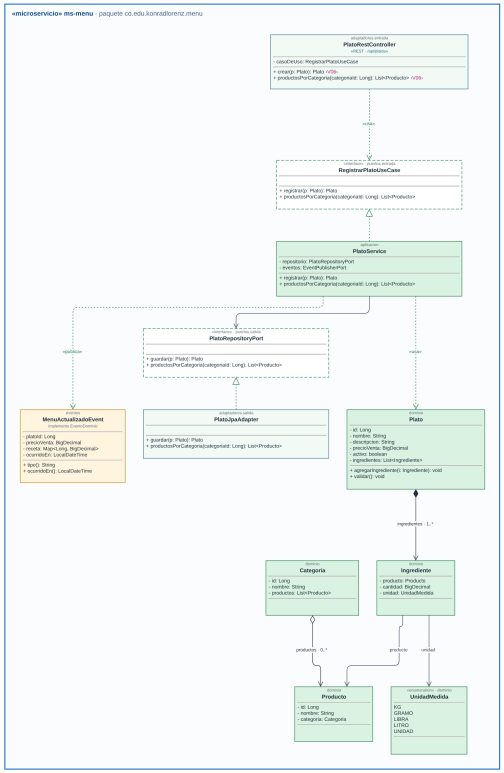


Figura A2 · Página 4b · pedidos, menú e inventario

Nivel 4 (C4) · Código · pedidos, menú e inventario

Diagrama de clases UML generado del mismo modelo que el esqueleto Java (paquetes `co.edu.konradlorenz.<servicio>.<capa>`). Los métodos marcados «Vxx» atienden esa ventana de los mockups; en magenta, el patrón de diseño que cumple cada clase.





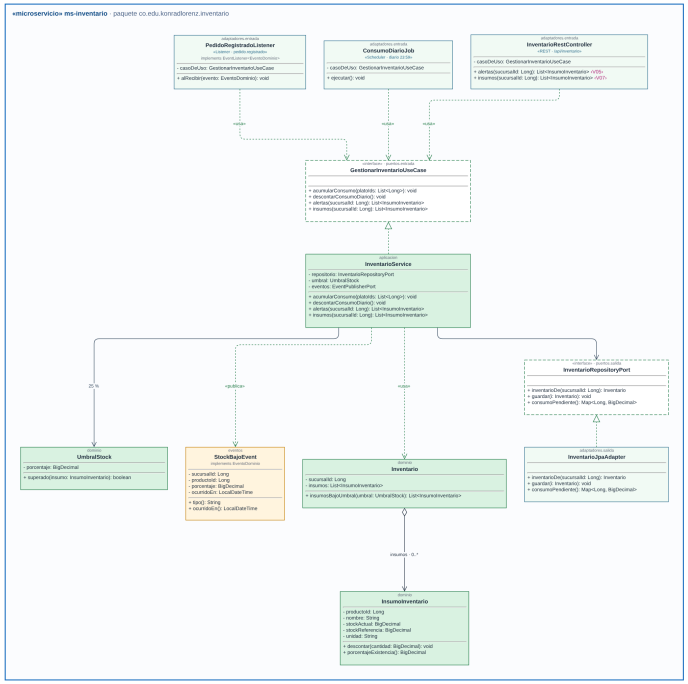
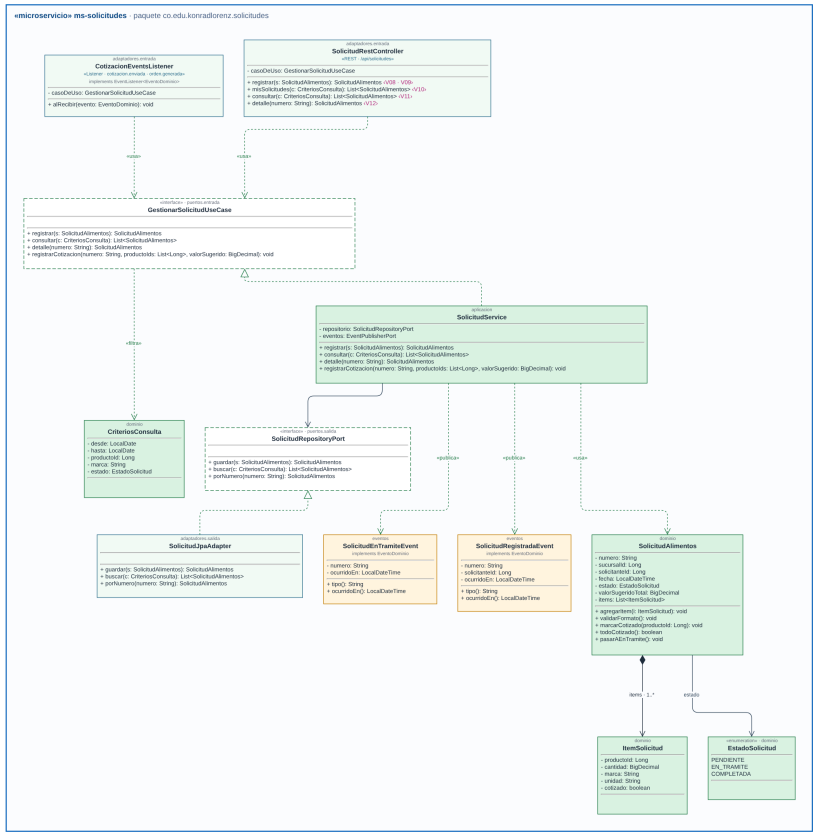


Figura A3 · Página 4c · solicitudes y cotizaciones

Nivel 4 (C4) · Código · solicitudes y cotizaciones

Diagrama de clases UML generado del mismo modelo que el esqueleto Java (paquetes `co.edu.konradlorenz.<servicio>.<capa>`). Los métodos marcados ‹Vxx› atienden esa ventana de los mockups; en magenta, el patrón de diseño que cumple cada clase.



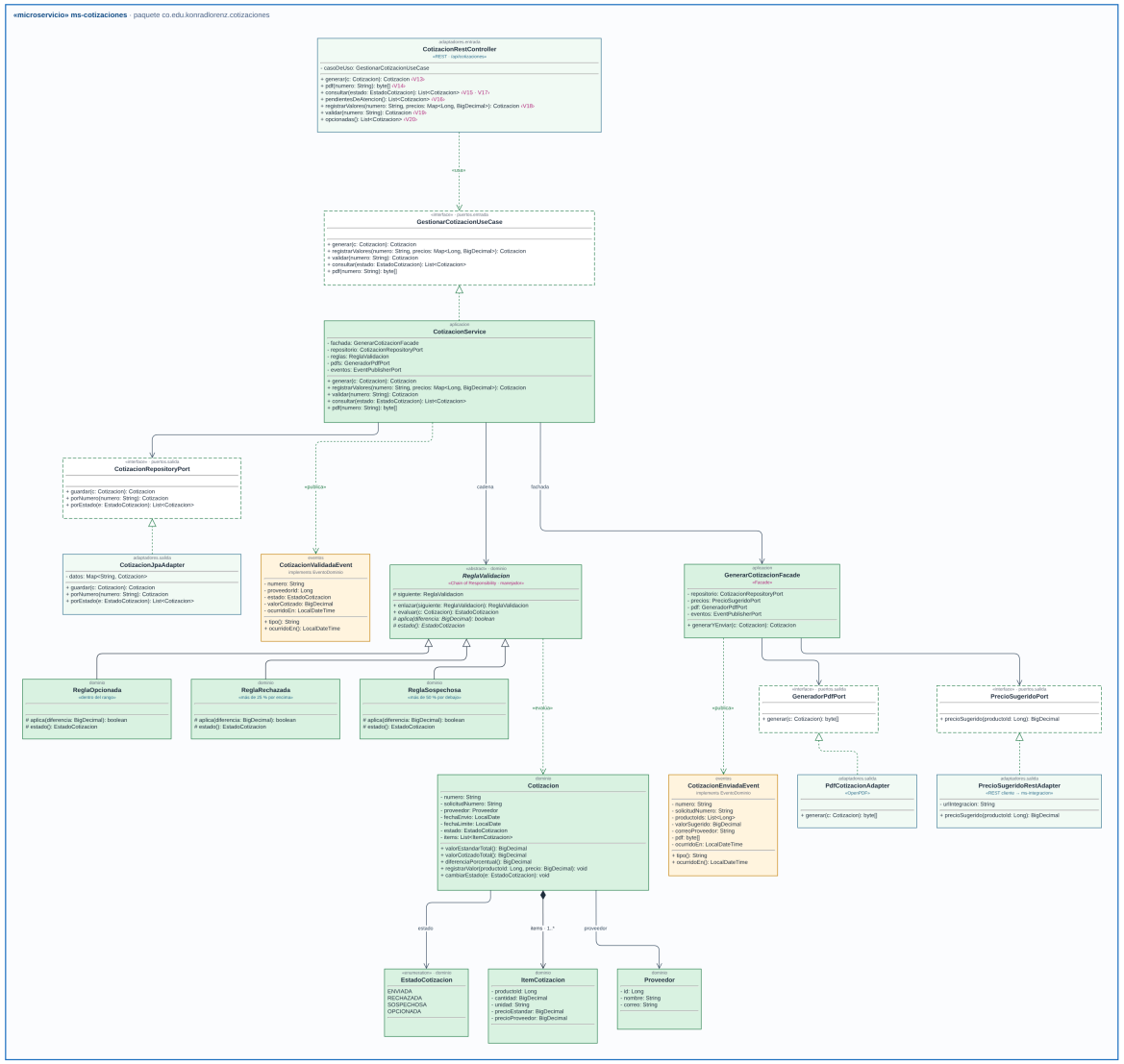
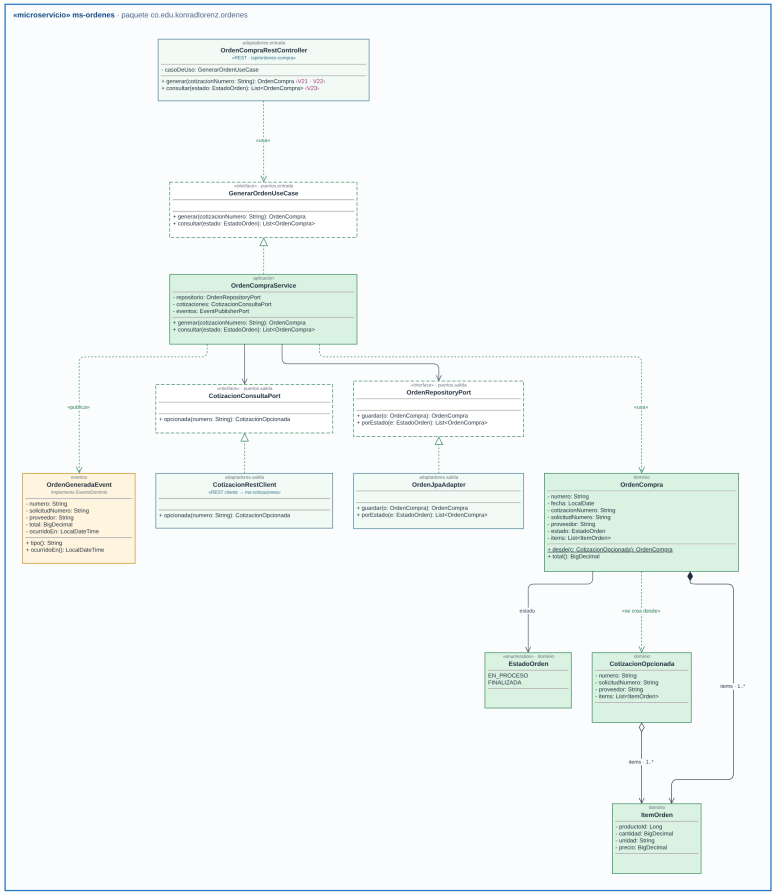


Figura A4 · Página 4d · órdenes y notificaciones

Nivel 4 (C4) · Código · órdenes y notificaciones

Diagrama de clases UML generado del mismo modelo que el esqueleto Java (paquetes `co.edu.konradlorenz.<servicio>.<capa>`). Los métodos marcados ‹Vxx› atienden esa ventana de los mockups; en magenta, el patrón de diseño que cumple cada clase.



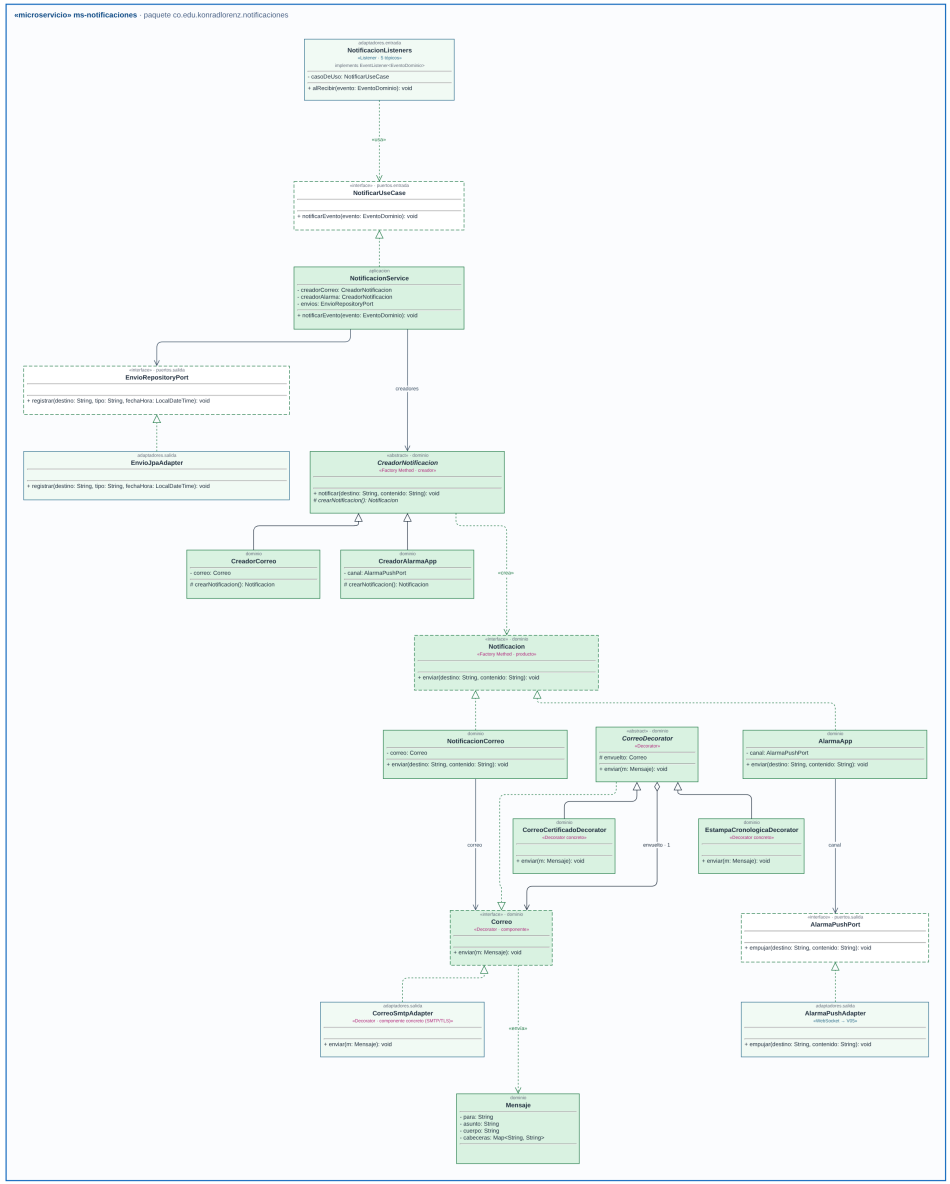


Figura A5 · Página 4e · integración, administración y analítica

Nivel 4 (C4) · Código · integración, administración y analítica

Diagrama de clases UML generado del mismo modelo que el esqueleto Java (paquetes `co.edu.konradlorenz.<servicio>.<capa>`). Los métodos marcados «Vxx» atienden esa ventana de los mockups; en magenta, el patrón de diseño que cumple cada clase.

